컴퓨터 그래픽스 시험 대비 통합 요약본

2026년 2학기 | 중간·기말 대비 | 2026.10.25

이론의 원리 → 공식의 조건 → 계산 과정 → 결과 검산

업로드된 강의 PDF **10개, 총 323쪽**를 하나로 묶은 학습용 요약본입니다. 이론 1~5와 OpenGL·GLSL·좌표계 변환 자료를 각각의 번호 순서에 맞춰 정리했습니다.

이 문서에 담은 것

- **핵심 개념과 알고리즘**: 정의, 작동 원리, 장단점, 서로 다른 개념의 비교.
- **계산 공식과 풀이**: 픽셀 결정, 행렬 합성, 클리핑, 조명, 버퍼 크기, 카메라와 화면 좌표.
- **시험 직전 복습**: 공식 모음, 자가 점검 문제, 정답과 풀이, 교수 자료의 표기 주의점.
- **함수 중심 보강 F01~F13**: 인자의 뜻 → 반환값 → 좌표 계산 → 호출 순서 → 코드 해석 문제. 이동·회전·신축·카메라·투영 함수를 집중 설명합니다.

읽는 방법

처음에는 각 절의 원리와 비교표를 읽고, 두 번째에는 계산 예제에서 공식을 고르는 이유를 확인하세요. 마지막에는 문제의 풀이를 가리고 직접 계산한 뒤 답을 대조하세요.

표시	의미
강의 기준	원본의 용어·기호·알고리즘을 우선합니다.
보충	수학적 연결이나 계산 연습을 위해 추가한 설명입니다.
정정·주의	원본의 오타·모호한 조건·일반화의 한계를 구분합니다.

실제 시험 범위의 제외 단원과 출제 비중은 제공되지 않았습니다. 이 문서는 제공된 자료 전체를 다룹니다. 풀이 예제는 강의 예제 또는 별도 표시한 보충 예제입니다.

읽기 지도와 출처 체계

10개 자료의 내용을 교차 참조하기 위한 안내

절	내용	원본 자료
01~02	개요·시스템, 계산 준비	[T1]
03~07	선·원·영역 채우기·안티 앨리어싱	[T2]
08~10	2차원 기하 변환·합성·반사	[T3]
11~14	윈도우·뷰포트·클리핑	[T4]
15~18	조명·쉐이딩·광선 추적	[T5]
19~23	OpenGL·GLSL·셰이더·그리기	[G1], [G2a]
24~25	VBO·VAO·EBO와 바이트 계산	[G2b]
26~30	3차원 변환·카메라·투영·가시성·모델	[G3a], [G3b]
31~35	공식·문제·해설·정정·원본 목록	전 자료 통합

시험 계산의 공통 순서

- **1. 조건**: 좌표계, 기울기 범위, 회전 방향, 기준점, 단위, 경계 포함 여부를 적습니다.
- **2. 식**: 알고리즘의 초기값과 갱신식을 먼저 쓰고 수치를 넣습니다.
- **3. 과정**: 현재 값과 다음 값을 분리합니다. 행렬은 정점 가까운 오른쪽부터 적용합니다.
- **4. 검산**: 끝점이 맞는지, 중심이 고정되는지, 밝기와 범위가 조건에 맞는지 확인합니다.

표기 약속

열벡터를 사용하며 p′=Mp로 씁니다. xk는 k번째 좌표, p(k+1)은 다음 판단매개변수입니다. 점과 판단매개변수에 같은 p가 쓰이면 문맥으로 구분합니다. 행렬은 도식 또는 행별 표기로 제시합니다.

각 절 위의 [T2] 8~13쪽처럼 표시한 범위는 **원본 PDF 쪽수**입니다. 이 문서의 쪽수와 다릅니다. 클릭 가능한 PDF 책갈피로 각 절을 이동할 수 있습니다.

01 개요와 그래픽스 시스템

[T1] 2~19쪽

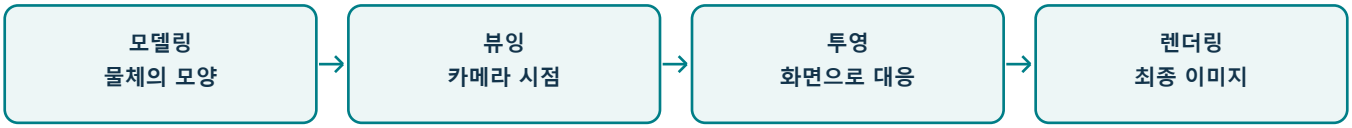
정의와 표현 방식

컴퓨터 그래픽스는 컴퓨터로 이미지를 생성·처리·표현하는 기술과 학문입니다. 강의는 가상 세계에서 이미지를 생성하는 그래픽스와, 이미 존재하는 영상을 변환·개선·분석하는 **영상처리**를 구분합니다. 실제 프로그램에서는 두 기능이 함께 쓰입니다.

구분	벡터 그래픽스	래스터 그래픽스
저장	점·선·곡선·도형의 수학적 정보	픽셀의 색상 배열
확대	다시 계산해 선명하게 표현 가능	픽셀이 드러나 계단·흐림 발생 가능
강점	로고·도면·문자·단순 기하 도형	사진·복잡한 색 변화·게임 출력 화면
비용	객체 복잡도에 영향	해상도·색상 깊이에 영향

벡터 파일이 항상 더 작은 것은 아닙니다. 객체의 수와 복잡도, 압축 형식에 따라 저장량이 달라집니다. 벡터 그림도 래스터 디스플레이에 표시할 때는 픽셀로 변환합니다.

3차원 이미지를 만드는 역할



- **모델링**: 와이어프레임, 다각형 표면, 솔리드, 3차원 스캔 등으로 모양을 표현합니다.
- **뷰잉**: 어느 위치와 방향에서 볼지 결정합니다. **투영**: 3차원 정보를 화면에 대응시킵니다.
- **렌더링**: 조명·재질·텍스처·가시성 등을 이용해 최종 2차원 이미지를 생성합니다. 넓게는 앞의 과정까지 포함해 쓰기도 합니다.

발전 흐름과 활용

초기 대화형 그래픽스·Sketchpad → 1970년대 3D·셰이딩·프랙털 → 1980년대 PC·GUI·CAD → 1990년대 3D API·GPU·3D 영화 → 실시간 GPU 렌더링, PBR, VR/AR, 광선 추적, AI 그래픽스로 발전했습니다. 게임, 영화, 설계, 의료 시각화, 시뮬레이션에 활용합니다.

02 하드웨어·버퍼와 계산 준비

[T1] 12~19쪽; 벡터·행렬 기초와 용량 계산은 보충

구성	역할과 예
입력 장치	2D: 마우스·태블릿·조이스틱·라이트 펜. 3D: 스캐너·디지타이저·모션 캡처.
출력 장치	프린터·플로터·3D 프린터, CRT·LCD·OLED 등 디스플레이.
GPU	많은 정점·프래그먼트의 유사한 계산을 병렬로 처리. 셰이더를 실행.
소프트웨어	OpenGL·Direct3D 등 라이브러리/API 위에서 응용 프로그램이 동작.

방식	랜덤 스캔	래스터 스캔
원리	도형의 선을 따라 그림	픽셀을 주사선 순서로 출력
기억 내용	선분 등 디스플레이 명령	픽셀 값을 담는 프레임 버퍼
장단점	선형 도면에 적합, 복잡한 사진에는 불리	사진·게임·색 채움에 적합, 저장량·계단 현상 발생

프레임 버퍼와 더블 버퍼

컬러 버퍼는 색상, **깊이 버퍼**는 가시성 판단용 깊이, **스텐실 버퍼**는 영역 제한용 값을 저장합니다. 더블 버퍼는 보이는 front와 그리는 back을 나눠 완성된 화면을 교환합니다. VSync는 교환을 디스플레이 갱신과 맞추는 별도 개념입니다.

저장량(byte) = 가로 × 세로 × 픽셀당 bit ÷ 8

보충 예제: 800×600, RGBA 8bit씩이면 컬러 버퍼 하나는 800×600×4=**1,920,000 byte**. 두 개는 3,840,000 byte 3.66 MiB입니다. 압축·정렬·추가 버퍼는 제외한 계산입니다.

이후 계산에 필요한 수학

a·b = ax bx + ay by + az bz
|a| = √(ax²+ay²+az²), normalize(a) = a/|a| (a≠0)
a·b = |a||b|cosθ, 라디안 = 도 × π/180

내적은 두 방향이 얼마나 같은지 나타내는 수입니다. **외적** a×b는 둘에 수직인 방향이며 순서를 바꾸면 부호가 바뀝니다. 행렬-벡터 곱은 각 행과 벡터의 내적을 결과 성분으로 만듭니다.

03 점·선과 DDA 알고리즘

[T2] 3~7쪽

기하학적 선과 화면의 픽셀

수학의 선은 연속적이지만 화면은 정수 격자입니다. **래스터화**는 연속적인 도형에 대응하는 픽셀/프래그먼트를 정하는 과정입니다. 점은 좌표 외에도 크기·색·명암·모양을, 선은 굵기·유형·색·끝 모양을 가집니다.

Δx=xb-xa, Δy=yb-ya
m=Δy/Δx, c=ya-mxa, y=mx+c

절대 좌표는 양 끝점을 직접 지정하고, 상대 좌표는 시작점과 변화량 (Δx,Δy)을 지정합니다. Δx=0인 수직선은 m을 계산하지 않고 x가 일정한 선으로 처리합니다.

DDA: 일정한 증분을 누적한다

- **|m|≤1**이고 x가 증가하는 경우: x를 1 늘리고 y를 m 늘립니다. |m|>1이면 y 방향을 기준으로 x를 1/m만큼 바꾸며 방향 부호를 반영합니다.
- 실수 좌표를 계속 유지하고, **출력할 때만 반올림**합니다. 반올림한 y를 다음 계산의 누적값으로 쓰면 다른 선이 됩니다.

보충 일반형: steps=max(|Δx|,|Δy|)
xinc=Δx/steps, yinc=Δy/steps
(x,y) ← (x+xinc, y+yinc), 출력은 (round(x),round(y))

steps=0이면 점 하나입니다. 일반형은 음의 기울기와 역방향도 다룹니다. 반올림 동률 규칙은 문제에서 지정한 방식을 따릅니다.

보충 예제: (1,1)에서 (5,3)

k	실수 좌표	출력 픽셀
0	(1,1)	(1,1)
1	(2,1.5)	(2,2)
2	(3,2)	(3,2)
3	(4,2.5)	(4,3)
4	(5,3)	(5,3)

이 예제는 양수의 .5를 올리는 반올림으로 계산했습니다.

장점: 단순하고 이해하기 쉽습니다. **단점**: 실수 누적·반올림이 필요하고 오차가 생길 수 있습니다. 다음 브레즌햄 알고리즘은 정수 판단식을 사용합니다.

04 브레즌햄 선: 판단식과 추적

[T2] 8~13쪽 | 강의 예제 (1,1)~(10,7)

두 후보 중 실제 선에 가까운 픽셀을 고른다

강의의 조건은 **0≤m≤1, x가 증가**하는 경우입니다. 현재 (xk,yk)에서 다음 후보는 E=(xk+1,yk), NE=(xk+1,yk+1)입니다. 실제 선까지의 아래·위 거리 차이를 정수식으로 바꾸면 판단매개변수 pk를 얻습니다.

pk=Δx(d1-d2), p1=2Δy-Δx
pk<0: E 선택, p(k+1)=pk+2Δy
pk≥0: NE 선택, p(k+1)=pk+2(Δy-Δx)

거리 차이에 양수 Δx를 곱해도 부호가 같으므로 가까운 후보 선택은 유지됩니다. 갱신식은 x가 항상 1 증가하고 y가 0 또는 1 증가한다는 사실에서 나옵니다. **pk=0은 강의에서 NE를 선택**합니다.

예제: Δx=9, Δy=6 → p1=3, C1=12, C2=-6.

현재 점	pk	선택	다음 점	다음 p
(1, 1)	3	NE	(2, 2)	-3
(2, 2)	-3	E	(3, 2)	9
(3, 2)	9	NE	(4, 3)	3
(4, 3)	3	NE	(5, 4)	-3
(5, 4)	-3	E	(6, 4)	9
(6, 4)	9	NE	(7, 5)	3
(7, 5)	3	NE	(8, 6)	-3
(8, 6)	-3	E	(9, 6)	9
(9, 6)	9	NE	(10, 7)	3

장점: 반복에서 정수 덧셈과 비교를 사용합니다. **주의**: 음의 기울기나 가파른 선에 위 식을 그대로 대입하지 않습니다. 축 교환·부호·대칭으로 기준 구간에 대응시킵니다.

05 원과 브레즌햄 원 그리기

[T2] 14~17쪽 | 강의 예제 중심 (0,0), 반지름 6

$(x-xc)^2+(y-yc)^2=r^2$
$x=xc+r\cos\theta$, $y=yc+r\sin\theta$

직교 좌표는 x,y로 위치를, 극 좌표는 거리 r과 각도 θ로 위치를 나타냅니다. 삼각함수로 원의 점을 만들 수 있지만 매번 계산하는 비용이 있습니다. 브레즌햄은 정수 판단식과 원의 대칭을 이용합니다.

45~90도 구간을 계산하고 8방향으로 복제

중심에 대한 **상대좌표 (0,r)**에서 시작해 x≤y 동안 진행합니다. 다음 후보는 (x+1,y)와 (x+1,y-1)입니다. 계산을 끝낸 뒤 중심 좌표를 더합니다.

$p1=3-2r$
$pk<0$: $(xk+1,yk)$, $p(k+1)=pk+4xk+6$
$pk\geq 0$: $(xk+1,yk-1)$, $p(k+1)=pk+4(xk-yk)+10$

갱신식에는 이동 전 xk,yk를 사용합니다. 반지름 6이면 p1=-9입니다.

현재 점	pk	다음 후보 선택	다음 p
(0, 6)	-9	(1, 6)	-3
(1, 6)	-3	(2, 6)	7
(2, 6)	7	(3, 5)	1
(3, 5)	1	(4, 4)	3
(4, 4)	3	(5, 3)	13

실제로 대칭 확장하는 구간의 점은 **(0,6),(1,6),(2,6),(3,5),(4,4)**입니다. 다음 (5,3)은 x>y라서 새로 확장하지 않습니다.

대칭점: (±x,±y), (±y,±x)
최종 위치: 각 대칭점에 (xc,yc)를 더한다.

예: (3,5)에서 (3,5),(-3,5),(3,-5),(-3,-5),(5,3),(-5,3),(5,-3),(-5,-3)을 얻습니다. x=0 또는 x=y에서는 중복점이 있으므로 서로 다른 점의 수를 셀 때 중복을 제거합니다.

06 영역 연결·내부 판정·시드 채우기

[T2] 18~25쪽

4방향과 8방향 연결

4방향은 상하좌우, **8방향**은 대각선을 포함합니다. 대각선으로만 닿은 두 픽셀은 4방향으로는 분리되고 8방향으로는 연결됩니다. 경계와 내부에 같은 연결 규칙을 쓰면 모서리에서 새는 등 모순이 생길 수 있습니다. 강의는 8방향 경계에 4방향 내부 채우기를 대응시킵니다.

내부 판단	계산 규칙	차이
홀짝 규칙	경계 교차 횟수 홀수: 내부, 짝수: 외부	방향을 보지 않아 단순
접기회수 규칙	강의 기준 아래 방향 +1, 위 방향 -1. 합≠0: 내부	모서리 방향에 따라 중첩 영역 판정 가능

보충 예: 같은 방향 경계를 두 번 교차하면 홀짝 규칙에서는 외부, 접기회수의 합이 +2라면 내부입니다. 꼭짓점을 정확히 지날 때는 교차를 중복 계산하지 않는 규칙이 필요합니다.

시드 채우기: 내부의 한 픽셀에서 퍼져 나간다

알고리즘	칠할 조건	멈추는 이유
Flood Fill	현재 색이 기준색 bgColor와 같음	다른 색을 만나면 멈춤
Boundary Fill	경계색도 아니고 이미 채운 색도 아님	경계 또는 방문한 픽셀을 만나면 멈춤

• **진행**: 시드를 스택/재귀 호출에 넣기 → 현재 조건 확인 → 색 변경 → 이웃 4개 또는 8개 검사.

• **Flood Fill 주의**: 기준색과 채우기색이 같으면 재방문을 막지 못할 수 있습니다. 같으면 즉시 종료하거나 방문 표시를 둡니다.

• **Boundary Fill 주의**: 경계가 열려 있거나 경계색 판정이 틀리면 바깥까지 채웁니다. fillColor 검사로 반복 방문을 막습니다.

• **장단점**: 이미 그려진 래스터 이미지에서 사용하기 쉽지만 넓은 영역에서 재귀 깊이·메모리가 커질 수 있습니다. 보충: N개 픽셀을 각 한 번 처리하면 O(N), 저장 공간은 최악 O(N)입니다.

07 주사선 채우기와 안티 앨리어싱

[T2] 26~34쪽

Y-X 다각형 주사선 알고리즘

도형의 벡터 정보를 이용해 각 수평 주사선과 변의 교차점을 구한 뒤 내부 구간을 채웁니다. **EL**은 전체 에지 목록, **AEL**은 현재 주사선에 걸리는 활성 에지 목록입니다. 이웃 주사선에서 에지가 크게 바뀌지 않는 **일관성(coherence)**을 이용합니다.

• 1. 에지를 시작 y의 오름차순으로 EL에 정렬합니다. 2. 현재 y에서 끝난 에지를 제거하고 새 에지를 AEL에 넣습니다. 3. 교차점 x를 정렬해 짝지어 채웁니다. 4. y를 증가시킵니다.

교차점 x=x1+(y-y1)(x2-x1)/(y2-y1)
다음 주사선: x(y+1)=x(y)+1/m (비수평 에지)

보충 예: 삼각형 (1,1),(5,1),(3,5)의 y=3에서는 왼쪽 변 x=2, 오른쪽 변 x=4이므로 구간 [2,4]를 채웁니다. 수평 에지와 공유 꼭짓점의 중복은 별도로 처리합니다. 구현에서는 흔히 ymin≤y<ymax를 사용하지만 강의 예제의 끝점 포함 방식과 구분해야 합니다.

앨리어싱과 안티 앨리어싱

앨리어싱은 연속적인 모양을 부족한 샘플로 표현할 때 다른 모양처럼 보이는 현상입니다. 선의 계단, 움직일 때의 경계 떨림이 예입니다. 경계 픽셀의 덮인 비율을 반영하면 시각적으로 부드러워집니다.

기법	원리	장점 / 한계
SSAA	픽셀 내부 여러 위치를 셰이딩하고 평균	다양한 앨리어싱 완화 / 계산·메모리 비용 큼
MSAA	여러 커버리지·깊이 샘플, 보통 셰이딩 결과 공유	도형 경계에 효율적 / 셰이더 내부 고주파에는 한계
FXAA·MLAA	완성 이미지의 경계를 찾아 후처리	비교적 저렴 / 세부 표현이 흐려질 수 있음

보충 단순 평균: C=αC도형+(1-α)C배경
α=덮인 샘플 수/전체 샘플 수

검정 배경 위 흰 도형이 4개 샘플 중 3개를 덮으면 C=0.75입니다. 이는 같은 가중치·선형 색 값의 단순 계산이며 실제 색 공간 처리 등은 별도입니다.

08 2차원 이동·신축·회전

[T3] 3~10쪽

기하 변환은 도형을 이루는 점들을 새 위치로 바꾸는 연산입니다. 이동은 위치를, 신축은 크기를, 회전은 방향을 바꿉니다. 신축과 회전은 기준점이 중요합니다.

변환	좌표식	기본 기준
이동	x′=x+tx, y′=y+ty	모든 점에 동일한 이동량
신축	x′=sx x, y′=sy y	원점. 위치도 배율에 따라 바뀜
회전	x′=x cosθ-y sinθ y′=x sinθ+y cosθ	원점. 양의 θ는 반시계

sx,sy>1이면 해당 축 확대, 0<s<1이면 축소, s=1이면 유지, s<0이면 축 방향 반전도 포함합니다. 한 축 배율이 0이면 그 축 정보가 사라집니다.

동차 좌표: 이동까지 행렬 곱으로 통일

2차원 점 (x,y)를 **(x,y,1)**로 확장하면 모든 기본 변환을 3×3 행렬로 쓸 수 있습니다. 방향벡터는 마지막 성분을 0으로 두어 이동의 영향을 받지 않게 합니다.

$$\mathbf{T}(t_x,t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{S}(s_x,s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

회전식은 x=r cosφ, y=r sinφ에서 각을 φ+θ로 바꾸고 덧셈정리를 전개하면 얻습니다. 검산: (1,0)을 반시계 90도 회전하면 (0,1)입니다.

09 합성 변환과 임의 기준점

[T3] 6~12쪽 | 강의 예제와 보충 검산

먼저 A, 다음 B이면 p′=BAp
M중심회전=T(x0,y0)R(θ)T(-x0,-y0)
M중심신축=T(x0,y0)S(sx,sy)T(-x0,-y0)

기준점을 원점으로 옮기고, 원점에서 변환한 뒤, 원래 위치로 돌려놓습니다. 식은 오른쪽부터 읽습니다. 기준점 자체를 대입했을 때 움직이지 않으면 좋은 검산입니다.

행렬 곱의 성질

- **결합법칙**: (AB)C=A(BC). 여러 변환을 하나의 행렬로 미리 결합할 수 있습니다.
- **일반적으로 교환 불가**: AB≠BA. 같은 축의 이동끼리, 2D 원점 회전끼리 등 일부 조합만 교환됩니다.
- 이동 두 번은 이동량의 합, 신축 두 번은 배율의 곱, 같은 중심의 2D 회전 두 번은 각도의 합입니다.

강의 예제: 중심 (0,1)에서 90도 회전 후 2배 신축

삼각형 A=(5,7), B=(10,5), C=(8,10). 마지막 신축은 별도 기준점이 없으므로 **원점 기준**으로 해석합니다. 회전 중심과 신축 중심이 같은지는 문제에서 확인해야 합니다.

M=S(2,2)T(0,1)R(90°)T(0,-1)
회전 결과: (x,y) → (1-y,x+1)
최종 결과: (x,y) → (2-2y,2x+2)

점	중심 이동	90도 회전	제자리 이동	원점 2배
A(5,7)	(5,6)	(-6,5)	(-6,6)	(-12,12)
B(10,5)	(10,4)	(-4,10)	(-4,11)	(-8,22)
C(8,10)	(8,9)	(-9,8)	(-9,9)	(-18,18)

순서가 달라지는 보충 예

점 (1,0)에 원점 90도 회전 후 x로 2 이동: TRp=(2,1). 반대로 먼저 이동 후 회전: RTp=(0,3). 회전이 물체의 방향만 바꾸는지, 원점 주위의 위치까지 바꾸는지 구분하세요.

10 반사·밀림과 변환 분류

[T3] 13~18쪽

반사 기준	결과	기억법
x축	(x,-y)	위아래 바꿈
y축	(-x,y)	좌우 바꿈
원점	(-x,-y)	두 부호 바꿈
y=x	(y,x)	두 성분 교환
y=-x	(-y,-x)	교환 후 두 부호 바꿈

y=mx+b에 대한 반사, θ=atan(m):
M=T(0,b)R(θ)Fx R(-θ)T(0,-b)

직선을 원점을 지나는 상태로 이동 → x축에 겹치게 회전 → x축 반사 Fx → 회전 복구 → 이동 복구입니다. 수직선은 이 기울기 표현과 별도로 처리합니다.

강의 예제 y=x+2: (x,y) → **(y-2,x+2)**. 따라서 (0,5)→(3,2), (8,2)→(0,10). 원래 점과 반사점의 중점이 직선 위인지 검산할 수 있습니다.

밀림(Shearing)

x방향: x′=x+hx y, y′=y
y방향: x′=x, y′=y+hy x

x방향 밀림에서는 x축(y=0)이 고정되고 x축에서 멀수록 더 밀립니다. 예: hx=2이면 (1,3)→(7,3). 각도와 길이는 일반적으로 유지되지 않습니다.

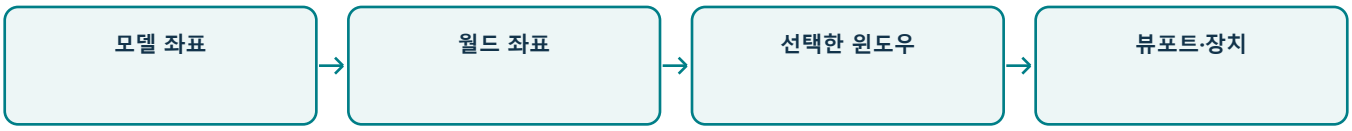
유형	보존 성질	대표 변환
강체	거리와 각도	이동·회전
유사	각도·모양, 길이는 같은 배율	이동·회전·균등 신축
선형	벡터 덧셈과 스칼라 곱, 원점	회전·신축·반사·밀림
어파인	직선·평행성·한 직선 위 길이의 비	선형 변환 + 이동
원근/사영	직선, 원근 관계 표현	원근 투영 등

정정: 비균등 신축은 일반적으로 유사 변환이 아닙니다. 어파인 변환은 일반적인 두 길이의 비를 모두 보존한다는 뜻이 아닙니다. 퇴화 변환에서는 일부 기하 성질에 추가 조건이 필요합니다.

11 윈도우와 뷰포트 매핑

[T4] 3~8쪽

윈도우는 월드에서 무엇을 보여줄지 선택한 영역입니다. **뷰포트**는 선택한 내용을 출력 장치의 어디에 보여줄지 지정한 영역입니다. 운영체제의 창과 구분하세요.



sx=(xvR-xvL)/(xwR-xwL)
sy=(yvT-yvB)/(ywT-ywB)
xv=xvL+(xw-xwL)sx
yv=yvB+(yw-ywB)sy

윈도우의 왼쪽 아래를 원점으로 옮기고, 크기를 뷰포트에 맞춘 뒤, 뷰포트 위치로 옮기는 식입니다. **M=T(xvL,yvB)S(sx,sy)T(-xwL,-ywB)**로 합성할 수 있습니다.

강의 예제

윈도우 (1,0)~(11,10), 뷰포트 (10,10)~(80,60)이면 sx=70/10=7, sy=50/10=5입니다.

xv=10+7(xw-1), yv=10+5yw
(2,3) → (17,25), (8,7) → (59,45)

두 점으로 지정한 사각형의 나머지 꼭짓점도 같은 식으로 계산합니다. sx≠sy이므로 모양의 가로세로 비율은 달라집니다.

효과	무엇을 바꾸는가
Zooming	뷰포트가 고정일 때 윈도우를 줄이면 같은 화면에 좁은 영역이 크게 보임
Panning	윈도우의 위치를 이동해 보는 영역을 옮김
Multi viewport	한 화면 안에 여러 뷰포트를 두어 서로 다른 시점·영역을 표시

클리핑의 시작

클리핑은 출력 영역 밖의 객체 또는 일부를 제거합니다. 점은 $x_L \le x \le x_R$이고 $y_B \le y \le y_T$일 때 남깁니다. 월드에서 먼저 자른 후 매핑하거나 매핑 후 뷰포트에서 자를 수 있습니다.

12 Cohen-Sutherland 선 클리핑

[T4] 9~12쪽

9개 영역을 4비트 코드로 표현

왼쪽 위 1001	위 1000	오른쪽 위 1010
왼쪽 0001	내부 0000	오른쪽 0010
왼쪽 아래 0101	아래 0100	오른쪽 아래 0110

강의 비트 약속은 **왼쪽 0001, 오른쪽 0010, 아래 0100, 위 1000**입니다. 모서리 밖에서는 비트를 OR로 합칩니다. 경계 위의 점은 내부로 봅니다.

(code1 OR code2)=0 → 전체 수락
(code1 AND code2)≠0 → 전체 거부
나머지 → 경계 교차점으로 끝점을 바꾸고 반복

AND가 0이 아니면 두 끝점이 같은 바깥 반평면에 있으므로 선분 전체가 밖입니다. **AND=0만으로 교차를 확정할 수는 없습니다.** 한 끝점이 내부이면 적어도 일부는 남지만 교차점 계산이 필요합니다.

교차점 계산

수직 경계 x=a: y=y1+m(a-x1)
수평 경계 y=b: x=x1+(b-y1)/m
보충 안전식: P(u)=P1+u(P2-P1)

수직선·수평선에서는 0으로 나누지 않도록 해당 경계와의 교차 여부를 따로 처리합니다.

강의 예제: 윈도우 [-10,10]×[-10,10]

P1=(-15,-15), P2=(12,8), 변화량=(27,23). 초기 코드는 0101,0010이며 AND=0입니다. 아래 경계 y=-10에서는 u=5/23, x=-210/23 -9.1304. 오른쪽 경계 x=10에서는 u=25/27, y=170/27 6.2963입니다.

남는 선분: (-210/23,-10) ~ (10,170/27)

장점: 완전 수락·거부를 비트 연산으로 빠르게 판정합니다. **단점**: 부분 교차에서 끝점과 코드를 반복 갱신하며 교차점을 여러 번 계산할 수 있습니다.

13 Liang-Barsky 선 클리핑

[T4] 13~20쪽

선분을 **P(u)=P1+u(P2-P1), 0≤u≤1**로 나타내고 네 경계 부등식을 모두 만족하는 u 구간을 구합니다. 교차점을 반복 수정하는 대신 진입·이탈 구간을 좁힙니다.

경계	pk	qk
왼쪽	-dx	x1-xmin
오른쪽	dx	xmax-x1
아래	-dy	y1-ymin
위	dy	ymax-y1

각 경계: pk u≤qk
pk<0: u≥qk/pk → 진입 하한을 크게
pk>0: u≤qk/pk → 이탈 상한을 작게
pk=0: qk<0이면 거부, qk≥0이면 추가 제한 없음

초기 ustart=0, uend=1. 모든 음수 pk의 비율 중 최댓값과 0을 비교해 시작을, 모든 양수 pk의 비율 중 최솟값과 1을 비교해 끝을 구합니다. **ustart>uend면 거부**, 같으면 한 점만 남습니다.

강의 예제: 윈도우 (5,5)~(10,10), P1=(3,8), P2=(12,2)

경계	pk	qk	qk/pk	제한
왼쪽	-9	-2	2/9	u≥2/9
오른쪽	9	7	7/9	u≤7/9
아래	6	3	1/2	u≤1/2
위	-6	2	-1/3	u≥-1/3

ustart=2/9, uend=1/2
P(2/9)=(5,20/3), P(1/2)=(15/2,5)

두 번째 강의 예제 (8,9)~(2,6)은 u∈[0,1/2]가 남아 **(8,9)~(5,7.5)**입니다. pk의 부호는 단순한 x/y 이동 방향이 아니라 각 경계 부등식에 대한 진입/이탈을 나타냅니다.

비교: Cohen-Sutherland는 영역코드·비트 연산, Liang-Barsky는 부등식·매개변수 범위가 핵심입니다. 후자는 불필요한 교차점 계산을 줄일 수 있습니다.

14 다각형 클리핑

[T4] 21~24쪽

Sutherland-Hodgman: 한 경계씩 잘라 출력 목록을 갱신

속이 빈 다각형은 각 선분을 자를 수 있지만, 채운 다각형은 잘린 뒤의 경계와 꼭짓점을 다시 만들어야 합니다. 한 경계에 대해 모든 변을 검사하고 나온 꼭짓점 목록을 다음 경계의 입력으로 사용합니다.

이전 S → 현재 P	출력	뜻
안 → 안	P	그대로 진행
안 → 밖	교점 I	경계를 만나며 나감
밖 → 안	교점 I, P	들어온 지점부터 살림
밖 → 밖	없음	현재 경계 안쪽 부분 없음

- 1. 마지막 꼭짓점을 첫 변의 이전 점 S로 둡니다. 2. 현재 P와 S의 내부 여부를 검사합니다. 3. 표에 따라 출력합니다. 4. S=P로 바꾸고 다음 변을 검사합니다.
- 왼쪽·오른쪽·아래·위 경계를 차례로 반복합니다. 경계 순서를 달리해도 유효한 최종 영역은 같아야 합니다.

보충 계산 예제

삼각형 A=(-1,1), B=(3,1), C=(1,3)을 x≥0에 대해 자릅니다. 이전 점 C에서 시작합니다.

변	경우	출력
C→A	안→밖	교점 (0,2)
A→B	밖→안	교점 (0,1), B=(3,1)
B→C	안→안	C=(1,3)

출력 다각형은 **(0,2),(0,1),(3,1),(1,3)**입니다. 변이 단순히 사라지는 것이 아니라 클리핑 경계를 따라 새 변이 생깁니다.

Weiler-Atherton과의 비교

Sutherland-Hodgman은 볼록한 클리핑 영역에 대해 경계별 처리가 단순합니다. 오목한 입력을 잘라 서로 분리된 결과를 표현할 때 한 목록만으로는 처리가 복잡해질 수 있습니다.

Weiler-Atherton은 교차점을 두 경계 목록에 넣고 진입·이탈을 구분해 필요한 경계를 번갈아 따라가며 닫힌 영역을 만듭니다. 여러 개로 분리되는 영역을 다룰 수 있지만 교차점 정렬과 추적이 복잡합니다. 원본의 경계 추적 설명은 축약되어 있어 이 원리 설명을 보충했습니다.

15 조명 모델: 주변·산란·거울 반사

[T5] 3~12쪽

조명 모델은 표면 한 점이 어떤 밝기·색으로 보이는지 계산합니다. 광원의 세기·색·위치, 표면의 색·반사 특성, 관찰자의 위치가 영향을 줍니다. 표면에서는 흡수·반사·투과/굴절이 일어납니다.

성분	직관	의존성
주변 조명	여러 방향의 간접광을 일정한 밝기로 근사	Ia와 주변 반사계수 Ka
산란 반사	무광택 표면이 여러 방향으로 고르게 반사	광원 세기·거리·표면 방향. 관찰 방향에는 무관한 Lambert 근사
거울 반사	특정 방향에서 보이는 반짝임	광원·법선·시선과 반사 방향, Ks와 광택 지수 n

주변: Iamb=Ka Ia
산란: Idiff=(Ip/d) Kd max(0,N·L)
거울: Ispec=(Ip/d) Ks [max(0,V·R)]^n
총합: I=Iamb+Idiff+Ispec

강의 기준: 거리 계수는 Ip/d입니다. 이상적인 점광원의 물리적 거리 감쇠 $1/d^2$와 구분하고, 시험에서 지정한 식을 우선합니다. d=0은 허용되지 않는 모델 조건입니다.

벡터의 뜻과 부호

• **N**: 표면 밖으로 향하는 법선. **L**: 표면에서 광원으로. **V**: 표면에서 관찰자로. **R**: 표면에서 나가는 반사광. 모두 단위 벡터로 계산합니다.

• **Lambert 코사인 법칙**: N·L=cosθ. 광원을 정면으로 향하면 1, 비스듬하면 작아집니다. 음수는 뒤쪽 빛이므로 직접 산란반사를 0으로 둡니다.

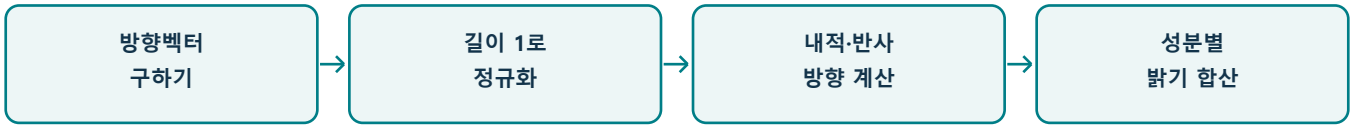
• V·R=cosΦ. Φ가 0에 가까울수록 강한 반짝임입니다. n이 클수록 반짝임의 범위가 좁아집니다. Ks는 크기, n은 주로 폭을 조절합니다.

실제 구현에서는 N·L≤0일 때 직접 거울반사도 차단하는 등 표면 앞뒷면 조건을 함께 둡니다. RGB 조명은 각 색 성분별로 같은 구조를 적용할 수 있습니다.

16 조명 계산과 검산

[T5] 7~12쪽 | 수치 예제와 반사벡터 유도는 보충

계산 순서



L=normalize(광원 위치-표면 위치)
V=normalize(관찰자 위치-표면 위치)
R=2(N·L)N-L (L은 표면→광원)

반사벡터는 L의 법선 방향 성분은 유지하고 접선 성분의 부호를 바꾼 형태입니다. GLSL reflect(I,N)의 I는 표면으로 들어오는 방향이므로 이 약속에서는 **reflect(-L,N)**에 해당합니다.

보충 예제

Ia=0.2, Ka=0.5, Ip=8, d=4, Kd=0.6, Ks=0.4, n=2. N=(0,0,1), L=(0,0.6,0.8), V=(0,0,1)이라 합시다. 세 벡터의 길이는 이미 1입니다.

계산	과정	결과
N·L	0+0+1×0.8	0.8
R	2×0.8(0,0,1)-(0,0.6,0.8)	(0,-0.6,0.8)
V·R	(0,0,1)·(0,-0.6,0.8)	0.8
주변	0.5×0.2	0.1
산란	(8/4)×0.6×0.8	0.96
거울	(8/4)×0.4×0.8^2	0.512
합계	0.1+0.96+0.512	1.572

1보다 큰 밝기는 수학적으로 계산될 수 있습니다. 문제에서 [0,1] 클램프를 요구하면 최종 출력은 1이며, 요구가 없다면 원래 계산값을 적습니다. 실제 HDR 처리와도 구분합니다.

자주 틀리는 부분

- 정규화하지 않은 방향벡터의 내적을 cosθ로 사용하지 않습니다. 예: (0,0,2)·(0,0,3)=6은 코사인이 아닙니다.
- n은 (V·R) 전체에 거듭제곱합니다. 주변 조명은 강의 식에서 거리 d로 나누지 않습니다.
- 거리만 두 배라면 Ip/d를 쓰는 직접광 성분은 절반입니다. $1/d^2$ 모델이라면 1/4이므로 두 모델을 혼합하지 않습니다.

17 다각형 쉐이딩과 보간

[T5] 13~16쪽

조명 모델은 한 점의 밝기 식이고, **쉐이딩**은 다각형 내부에서 어떤 값을 보간하고 어디에서 조명식을 계산할지 정하는 방식입니다. Phong이라는 이름이 두 곳에 쓰여도 같은 개념은 아닙니다.

방식	계산·보간 대상	장점	한계
균일/Flat	면마다 하나의 밝기·색	빠르고 다면체 표현에 적합	면 경계가 드러남
Gouraud	정점에서 조명 → 밝기/색 보간	부드럽고 내부 조명 계산이 적음	정점 사이의 좁은 highlight를 놓칠 수 있음
Phong	법선 보간 → 각 프래그먼트에서 조명	곡면·거울반사 표현이 좋음	계산량이 많음

선형 보간: 두 값 사이를 비율로 섞는다

I(t)=(1-t)I1+tI2, 0≤t≤1
t=(xp-x4)/(x5-x4)
Ip=[(x5-xp)/(x5-x4)]I4+[(xp-x4)/(x5-x4)]I5

Gouraud는 먼저 삼각형 양쪽 변에서 주사선의 밝기 I4,I5를 보간하고, 그 사이를 다시 x로 보간합니다. 보충 예: x4=2, x5=8, I4=0.2, I5=0.8이면 xp=5에서 t=1/2, **Ip=0.5**입니다.

Phong의 법선 보간

Nraw=(1-t)N4+tN5
N=normalize(Nraw), 그 뒤 조명식 계산

보간한 벡터의 길이는 보통 1이 아닙니다. N4=(1,0,0), N5=(0,1,0)의 중간은 (0.5,0.5,0)이고 정규화하면 (1/√2,1/√2,0)입니다.

Mach band는 밝기 변화의 경계가 더 두드러져 보이는 지각 현상입니다. 부드러운 보간·법선 기반 쉐이딩은 경계와 하이라이트의 부자연스러움을 줄이는 데 도움이 됩니다. 원근 장면의 속성 보간은 화면에서의 단순 선형 보간과 원근 보정을 구분합니다.

18 광선 추적과 래스터화 비교

[T5] 17~19쪽; [G1] 5~9쪽 | 광선 식은 보충

광선 추적법(Ray Tracing)의 원리

빛은 물체에서 반사·굴절을 반복합니다. 그 경로를 추적해 최종 색을 합산합니다. 실제 계산에서는 흔히 **카메라에서 픽셀을 지나 장면으로 광선**을 보내 가장 가까운 교점을 찾고, 그 점에서 광원·반사·굴절을 조사합니다.

광선: $P(t)=O+tD,\ t\geq 0$
$I=I_{amb}+I_{diff}+I_{spec}+I_{refl}+I_{refr}$

- **1차 광선**: 보이는 표면의 가장 가까운 교점 탐색. **그림자 광선**: 표면과 광원 사이를 가리는 물체가 있는지 검사.
- **반사·굴절 광선**: 다음 표면을 추적해 들어오는 색을 가중 합산합니다. 투명도와 재질에 따라 필요한 경로가 달라집니다.
- **종료**: 더 만나는 물체가 없거나, 광원을 만나거나, 반사·굴절 횟수 한도에 도달하거나, 기여도가 임계값 이하일 때입니다. 광원·배경을 만날 때의 처리는 모델에 따라 달라집니다.

비교	래스터화	광선 추적
진행 방향	도형을 화면에 투영해 프래그먼트 생성	화면/카메라 광선과 장면의 교차 탐색
가시성	깊이 버퍼 등으로 비교	광선의 가장 가까운 교점
반사·그림자	별도 기법이 필요할 수 있음	추가 광선의 교차로 자연스럽게 표현
강점	많은 도형의 실시간 처리에 효율적	반사·굴절·그림자 관계를 직접 모델링
비용	해상도·도형·프래그먼트 처리량	광선 수·교차 검사·경로 깊이

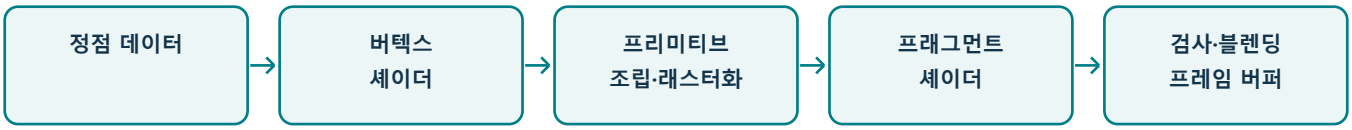
광선 추적은 사실적인 결과를 얻기 좋지만 계산량이 큽니다. 모든 광선 추적이 완전한 전역 조명을 계산하는 것은 아닙니다. 보충: 경로 추적(path tracing)은 빛의 전달을 확률적으로 샘플링하는 관련 방법이며 이번 자료의 핵심 범위를 넘어갑니다.

19 OpenGL과 그래픽스 파이프라인

[G1] 2~13쪽; [G2a] 2~6쪽

OpenGL은 GPU에 그래픽 작업을 요청하는 API 명세입니다. 창 생성·입력 처리는 별도 라이브러리가 담당합니다. OpenGL은 **상태 머신**이므로 활성 프로그램·버퍼·깊이 검사 등의 상태가 이후 명령에 영향을 줍니다.

구성	역할
GL / GLU	GL은 기본 렌더링 API. GLU는 전통적인 유틸리티·quadric 기능. 모던 Core 사용과 구분.
GLFW	창·OpenGL 컨텍스트·입력·이벤트·시간
GLEW 또는 GLAD	현재 컨텍스트에서 사용할 OpenGL 함수 주소 로딩
GLM	C++ 벡터·행렬·변환 수학. header-only 라이브러리
GLSL	GPU 셰이더를 작성하는 언어



- **버텍스 셰이더**: 정점 속성을 받아 위치·법선 등을 변환합니다. gl_Position에 클립 좌표를 출력합니다.
- **조립·클리핑·래스터화**: 점·선·삼각형 구성, 가시 영역 자르기, 도형이 덮는 샘플의 프래그먼트 생성. 클리핑은 셰이더가 직접 반드시 수행하는 작업이 아닙니다.
- **프래그먼트 셰이더**: 색·재질·조명 계산. 프래그먼트는 최종 픽셀 후보이며 검사에서 탈락하거나 다른 값과 합성될 수 있습니다.
- **선택 단계**: 테셀레이션은 메시 세분화, 지오메트리 셰이더는 프리미티브에서 새 도형을 생성·변경. 컴퓨트 셰이더는 별도 범용 GPU 계산 단계입니다.

고정 파이프라인	프로그래머블 파이프라인
정해진 변환·조명 기능의 상태를 설정. 단순하지만 효과가 제한적.	VBO/VAO와 직접 작성한 셰이더를 사용. 자유롭지만 데이터·프로그램 관리가 필요.

버전 맥락: 초기 1.x 고정 기능, 2.0 GLSL 도입, 3.x Core/Compatibility 분리와 모던 구조, 4.x 추가 단계·범용 계산. 강의의 구현 기준은 OpenGL 3.3 / GLSL 330 core입니다.

20 컨텍스트·렌더링 루프·이벤트

[G1] 14~44쪽

컨텍스트는 OpenGL의 실행 상태와 객체 관리 정보를 담은 환경입니다. 창을 만들었어도 현재 스레드에서 컨텍스트를 활성화해야 올바르게 OpenGL 함수를 호출할 수 있습니다.

glfwInit → glfwCreateWindow → glfwMakeContextCurrent → glewInit → 버퍼·셰이더 준비 → 렌더링 루프 → 정리

```
while (!glfwWindowShouldClose(window)) {
    // 입력/장면 상태 갱신
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    // 셰이더와 VAO 활성화, 그리기
    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

코드의 깊이 버퍼 클리어는 깊이 버퍼를 사용하는 경우입니다. 이벤트를 루프 앞에서 처리하는 구성도 가능합니다.

방식	폴링	콜백
동작	필요한 시점에 현재 상태 조회	이벤트가 처리될 때 등록 함수 호출
예	glfwGetKey, glfwGetMouseButton, glfwGetCursorPos	glfwSetKeyCallback, glfwSetMouseButtonCallback
적합	키를 누르고 있는 동안의 이동	클릭·눌림 순간·문자·창 크기 변경

• **glfwPollEvents**는 이벤트를 처리하고 즉시 돌아옵니다. **glfwWaitEvents**는 이벤트가 올 때까지 기다려 유휴 자원을 줄입니다. 게임 애니메이션과 UI에서 요구가 다릅니다.

• **KeyCallback**: 어떤 키인가. action은 PRESS/RELEASE/REPEAT, mods는 Shift·Control 등 상태. **CharCallback**: 어떤 문자가 입력되었는가. 텍스트 입력은 문자 콜백을 구분합니다.

• **마우스**: 버튼·이동·휠 콜백을 구분합니다. 드래그 변화량은 현재 좌표-이전 좌표이며 드래그 시작 프레임에는 기준 위치를 먼저 저장합니다.

• **Window Size**는 논리적 창 크기, **Framebuffer Size**는 실제 픽셀 크기입니다. HiDPI에서는 다를 수 있으므로 glViewport는 보통 프레임 버퍼 크기를 사용합니다.

21 시간·출력 상태와 프로그램 구성

[G1] 45~60쪽

Δt=currentTime-lastTime
position ← position+speed×Δt
angle=각속도×경과시간, 평균 FPS=프레임 수/시간

프레임마다 일정 거리만 더하면 기기 성능에 따라 이동 속도가 달라집니다. **초당 속도×경과 시간**을 사용하면 같은 시간에 같은 거리를 이동합니다. 보충 예: 3 unit/s에서 0.016 s가 지나면 0.048 unit 이동합니다.

더블 버퍼와 VSync

glfwSwapBuffers는 back 결과를 화면에 제시합니다. glfwSwapInterval(1)은 보통 한 번의 수직 갱신마다 교환하도록 요청하고, 0은 대기를 끕니다. 60Hz에서 동기화하더라도 매 프레임을 제시간에 만들지 못하면 항상 60FPS가 보장되지는 않습니다. 화면 찢김(tearing)은 갱신 중 서로 다른 프레임이 섞여 보이는 현상입니다.

함수	뜻	주의
glClearColor(r,g,b,a)	지울 때 사용할 색을 설정	이 함수만으로 지워지지는 않음
glClear(mask)	지정한 컬러·깊이·스텐실 버퍼 초기화	여러 비트는 |로 조합
glViewport(x,y,w,h)	NDC를 대응시킬 출력 영역	좌측 아래 시작, 실제 픽셀 크기
glfwSetWindowShouldClose	종료 요청 플래그 설정	다음 루프 조건에서 확인
glfwDestroyWindow / glfwTerminate	창·라이브러리 정리	필요한 GPU 자원도 정리

설치 자료를 이해하는 관점

- **헤더**는 선언, **.lib**는 링크, **.dll**은 실행 시 필요한 동적 라이브러리입니다. x64/x86 구성을 일치시켜야 합니다.
- 강의 예시 링크 대상은 opengl32.lib, glew32.lib, glfw3.lib입니다. GLM은 보통 헤더만 포함합니다. 설치 경로·버전 번호는 각 PC 환경에 따라 달라집니다.
- glColor3f·glRectf는 레거시 기능입니다. OpenGL 3.3 Core에서는 이 예제를 그대로 사용하지 않고 정점 속성과 셰이더로 그립니다. Compatibility 예제와 구분합니다.

OpenGL 타입의 크기는 GL 명세 기준으로 이해합니다. GLint는 32bit지만 C++ long은 플랫폼에 따라 다를 수 있으므로 무조건 같은 타입이라고 외우지 않습니다.

22 GLSL 타입·행렬·스위즐링

[G2a] 7~12쪽, 19쪽

타입	의미	예
스칼라	단일 수·논리값	float, int, uint, bool
벡터	같은 타입의 2~4성분	vec3, ivec4, uvec2, bvec3
행렬	열 개수 × 행 개수	mat3x2는 3열 2행
샘플러	텍스처에 접근하는 타입	sampler1D, sampler2D, sampler3D

double 계열은 GLSL 버전에 따라 지원 조건이 다릅니다. 강의의 330 core 예제는 float·vec·mat 중심으로 읽습니다. GLSL은 C 계열 문법, 함수 오버로딩·구조체·배열을 지원하지만 재귀 호출과 일반 C 포인터·동적 할당은 사용하지 않습니다.

생성자와 연산

vec3(2.0)=(2,2,2), mat3(1.0)=3×3 단위행렬
vec3끼리 * → 성분별 곱, mat끼리 * → 행렬 곱
mat2(1,2,3,4) = [1 3; 2 4]

행렬 생성자의 값은 **열 우선**으로 들어갑니다. M=mat2(1,2,3,4), v=(10,20)이면 Mv=(1×10+3×20,2×10+4×20)=**(70,100)**. 저장 순서와 실제 적용 순서(오른쪽부터)는 서로 다른 개념입니다.

스위즐링: 성분의 선택·재배열·복제

v=vec4(1,2,3,4)
v.xyxx=(1,2,1,1), v.wzy=(4,3,2)
xyzw / rgba / stpq는 각각 같은 성분을 가리킨다.

하나의 선택식 안에서 x와 g 같은 서로 다른 이름 집합을 섞지 않습니다. 읽기에서는 같은 성분을 반복할 수 있지만, 대입 대상 v.xx처럼 중복된 성분에 쓰는 것은 허용되지 않습니다.

함수	결과
mix(a,b,t)	(1-t)a+tb
step(edge,x)	x<edge이면 0, x≥edge이면 1
clamp(x,lo,hi)	범위 밖 값을 가까운 경계로 제한
length / distance / dot / cross / normalize	길이 / 거리 / 내적 / 외적 / 단위벡터
sin·cos·tan, pow·sqrt 등	삼각함수는 라디안, 거듭제곱·제곱근 등

23 한정자·셰이더 생성·그리기

[G2a] 13~38쪽

한정자/변수	역할
const	실행 중 수정하지 않는 상수
in / out	셰이더 단계의 입력 / 출력. 연결되는 인터페이스의 이름·타입을 맞춤
layout(location=n)	정점 속성 등 데이터 연결 위치를 명시
uniform	응용 프로그램이 설정하고 셰이더가 읽는 프로그램의 공통 값
gl_Position	버텍스 셰이더가 출력하는 vec4 클립 좌표
gl_VertexID	현재 정점 ID. DrawArrays는 first부터 진행, 인덱스 그리기에서는 인덱스 값에 대응
gl_FragCoord / gl_FrontFacing	프래그먼트의 윈도우 좌표 / 앞면에서 온 것인지 여부

객체를 만드는 순서

각 셰이더: glCreateShader → glShaderSource → glCompileShader
프로그램: glCreateProgram → glAttachShader → glLinkProgram
사용: glUseProgram → uniform 설정 → 그리기

컴파일은 glGetShaderiv(GL_COMPILE_STATUS), 링크는 glGetProgramiv(GL_LINK_STATUS)로 확인하고 각각 InfoLog를 읽습니다. 둘을 혼동하지 않습니다. 링크 후 필요 없는 셰이더 객체는 삭제 예약할 수 있습니다.

uniform 위치는 **GLint**로 받습니다. glGetUniformLocation의 -1은 찾지 못했거나 최적화로 비활성화되었음을 뜻할 수 있습니다. glUniform은 현재 활성 프로그램을 대상으로 합니다. uniform이 바뀌지 않으면 매 정점마다 다시 설정할 필요는 없습니다.

그리기	뜻
glDrawArrays(mode,first,count)	first부터 count개의 정점을 배열 순서로 사용
glDrawElements(mode,count,type,offset)	EBO의 count개 인덱스를 사용. offset은 바이트 위치
GL_POINTS / GL_LINES / GL_TRIANGLES	정점당 점 / 2개당 선 / 3개당 삼각형
LINE_STRIP / LINE_LOOP	연속 선 / 마지막과 처음도 연결
TRIANGLE_STRIP / TRIANGLE_FAN	n≥3에서 n-2개 삼각형. strip은 연결 방향에 주의

glPointSize는 점 크기, glLineWidth는 선 굵기, glPolygonMode는 GL_POINT/LINE/FILL 표현을 지정합니다. 프래그먼트 셰이더에서 나온 색이 그대로 최종 픽셀이 된다고 단정하지 않습니다.

24 VBO·VAO·EBO와 데이터 흐름

[G2b] 3~27쪽, 44~46쪽

객체	저장 내용	직관
VBO	정점의 위치·색·법선·텍스처 좌표 등 실제 데이터	재료 창고
VAO	각 속성을 어느 버퍼에서 어떤 형식으로 읽는지, 활성화 상태, EBO 연결	재료를 읽는 설명서
EBO	정점들을 사용할 순서를 나타내는 인덱스	재료 번호 목록

바인드(bind)는 이후 작업할 현재 대상을 정하는 것입니다. VAO에 정점 데이터 자체가 복사되는 것은 아닙니다. glVertexAttribPointer를 호출할 때 바인딩된 GL_ARRAY_BUFFER가 해당 속성에 연결됩니다.

위치 속성 하나의 준비 순서

```
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, bytes, data, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 12, (void*)0);
glEnableVertexAttribArray(0);
```

float가 4byte인 vec3 위치 배열 예입니다. 셰이더 입력은 layout(location=0) in vec3 position;로 연결할 수 있습니다.

• **glBufferData**: 버퍼 저장 공간과 초기 데이터를 지정합니다. STATIC/DYNAMIC/STREAM은 사용 패턴에 대한 힌트이며 자동 애니메이션 기능이 아닙니다.

• **glBufferSubData**: 이미 있는 버퍼의 offset부터 size 바이트를 교체합니다. offset·size를 정점 개수로 넣지 않습니다.

• **glVertexAttribPointer**: index, 성분 수, 타입, 정규화 여부, stride, 바이트 offset을 설정합니다. GL_FLOAT에서는 normalized 플래그로 길이 1 벡터를 만드는 것이 아닙니다.

정점 속성과 uniform의 구분

각 정점마다 다른 위치·색은 attribute로 전달합니다. 도형 전체에 공통인 변환 행렬·단색 등은 uniform으로 전달할 수 있습니다. 버텍스 셰이더 out 색은 래스터화 중 보간되어 프래그먼트 셰이더 in으로 들어갑니다.

25 stride·offset과 인덱스 계산

[G2b] 18~47쪽

두 속성을 한 VBO에 번갈아 저장

정점 형식: x y z r g b | x y z r g b | …
float=4byte → 정점 하나=6×4=24byte
위치: stride=24, offset=0
색상: stride=24, offset=3×4=12

stride는 같은 속성의 이번 정점 시작부터 다음 정점 시작까지의 바이트 간격입니다. 단순한 빈 공간 크기가 아닙니다. **offset**은 버퍼 시작에서 첫 속성이 시작되는 위치입니다.

0부터 센 정점	위치 시작	색상 시작
0	0	12
1	24	36
2	48	60

위치와 색상을 별도 VBO에 각각 촘촘히 넣으면 각 stride는 12, offset은 0입니다. glVertexAttribPointer의 전통 API에서 stride=0은 해당 속성이 촘촘히 이어졌다는 뜻입니다.

사각형을 삼각형 2개로 그리는 강의 예제

공유 정점 4개와 인덱스 (0,1,3),(1,2,3)을 사용합니다. glDrawElements의 count는 **6**이며 고유 정점 수 4나 삼각형 수 2가 아닙니다.

보충 용량 계산	위치만 vec3	위치 vec3+색 vec3
중복 정점 6개	6×12=72byte	6×24=144byte
고유 정점 4개	4×12=48byte	4×24=96byte
uint 인덱스 6개	6×4=24byte	6×4=24byte
VBO+EBO 합계	72byte	120byte

이 작은 위치 전용 예에서는 인덱스를 써도 합계가 같습니다. 재사용이 많거나 정점 속성이 커질수록 이득이 커질 수 있습니다. 인덱스 타입을 작게 정할 수 있으면 EBO도 줄어듭니다.

- **주의**: 위치가 같아도 면마다 법선·색·UV가 다르면 하나의 정점으로 공유할 수 없습니다. 공유 여부는 정점의 모든 필요한 속성이 같은지로 판단합니다.
- 인덱스 3개 뒤에서 시작하고 타입이 uint이면 바이트 offset=3×4=12입니다. DrawArrays의 first는 정점 단위, DrawElements의 offset은 바이트 단위입니다.

26 3차원 동차 좌표와 모델링 변환

[G3a] 2~27쪽

점은 위치이고 **벡터**는 크기와 방향입니다. 동차 좌표에서 점은 (x,y,z,1), 방향은 (x,y,z,0)으로 표현하면 하나의 4×4 행렬로 둘을 구분할 수 있습니다.

T·(x,y,z,1)=(x+tx,y+ty,z+tz,1)
T·(x,y,z,0)=(x,y,z,0)
(x,y,z,w), w≠0 → 실제 점 (x/w,y/w,z/w)

예: (2,4,8,2)와 (1,2,4,1)은 같은 실제 점입니다. 방향의 w=0을 원근 나누기에 넣어 점으로 해석하지 않습니다.

모델링 변환과 GLM 누적

모델 공간의 정점을 월드 공간의 위치·방향·크기로 배치합니다. GLM은 열벡터 관례에 맞춰 일반적으로 입력 행렬의 **오른쪽에 새 변환을 곱한 행렬**을 반환합니다.

```
glm::mat4 M(1.0f);
M = glm::translate(M, glm::vec3(2, 0, 0));
M = glm::rotate(M, glm::radians(90.0f), glm::vec3(0, 0, 1));
M = glm::scale(M, glm::vec3(2, 2, 2));
// M = T R S: 정점에는 신축 → 회전 → 이동
```

보충 검산: (1,0,0,1)→신축 (2,0,0,1)→z축 회전 (0,2,0,1)→이동 **(2,2,0,1)**. 함수 호출 순서와 실제 정점 적용 순서를 혼동하지 않습니다.

축 회전	좌표식: c=cosθ, s=sinθ
x축	x′=x, y′=cy-sz, z′=sy+cz
y축	x′=cx+sz, y′=y, z′=-sx+cz
z축	x′=cx-sy, y′=sx+cy, z′=z

x/y축 회전 좌표식은 오른손 좌표계 기준의 보충입니다. 강의는 z축 행렬과 glm::rotate를 중심으로 설명합니다. 회전축은 정규화하고 각은 라디안으로 전달합니다.

행렬 전송: glUniformMatrix4fv(location,1,GL_FALSE,glm::value_ptr(M)). GL_FALSE는 메모리를 임의로 뒤집지 않고 GLM의 열 우선 데이터를 사용한다는 뜻입니다.

26A 3차원 기본 변환 행렬

[G3a] 7~12쪽 | 열벡터 p=(x,y,z,1), 변환은 p′=Mp

이동은 마지막 열에 이동량, 신축은 주대각선에 배율을 둡니다. z축 회전은 2차원 회전식을 x,y 성분에 적용하고 z는 유지합니다. 아래 세 행렬은 점의 w=1을 그대로 유지하는 어파인 변환입니다.

이동 T =

$$\begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

신축 S =

$$\begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

z축 Rz =

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

행과 열을 확인하는 검산

- T의 첫 행과 점의 내적은 x+tx, 마지막 행과 점의 내적은 1입니다. 방향벡터는 w=0이므로 tx,ty,tz가 더해지지 않습니다.
- S는 원점 기준입니다. sx=sy=sz이면 균등 신축이고, 다르면 비균등 신축입니다. 음수 배율은 축 반전을 포함합니다.
- Rz(90도)에 (1,0,0,1)을 넣으면 (0,1,0,1)입니다. 회전 부호가 틀리면 시계방향 결과가 나옵니다.

일반 합성 M=TRS이면
p′=T(R(Sp)): 신축 → 회전 → 이동

27 카메라와 뷰잉 변환

[G3b] 3~9쪽 | 5쪽 뷰 행렬의 부호 정정 포함

뷰잉 변환은 월드의 점을 카메라 기준 좌표로 표현합니다. 기본 뷰 공간에서 카메라는 원점, 오른쪽은 +x, 위는 +y, 보는 방향은 **-z**입니다. 카메라가 오른쪽으로 가면 물체는 카메라 기준으로 왼쪽으로 이동합니다.

f=normalize(At-Eye)
r=normalize(f×Up), u=r×f
View=R T(-Eye)

Eye는 카메라 위치, At은 목표점, Up은 위쪽 기준입니다. f는 전방, r은 오른쪽, u는 실제 위쪽입니다. 카메라의 +z 축은 -f입니다. Eye=At이거나 f와 Up이 평행하면 축을 만들 수 없습니다.

View =

$$\begin{bmatrix} rx & ry & rz & -r\cdot Eye \\ ux & uy & uz & -u\cdot Eye \\ -fx & -fy & -fz & +f\cdot Eye \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

정정: 강의 5쪽 결과 행렬의 세 번째 행 마지막 항은 -f·Eye로 인쇄되어 있지만, R의 세 번째 행이 -f이므로 곱한 결과는 **+f·Eye**입니다. 카메라 위치를 대입해 원점이 되는지로 확인할 수 있습니다.

보충 예제: Eye=(0,0,5), At=(0,0,0), Up=(0,1,0)

계산	결과
f	(0,0,-1)
r=f×Up	(1,0,0)
u=r×f	(0,1,0)
View	x′=x, y′=y, z′=z-5
월드 원점 / 카메라 위치	(0,0,-5) / (0,0,0)

glm::lookAt(Eye,At,Up)이 이 뷰 행렬을 만듭니다. 카메라의 위치·방향을 바꾸는 행렬의 역변환이라는 관점으로 이해할 수 있습니다. 외적 순서를 뒤집으면 오른쪽 축이 반대가 됩니다.

28 직각 투영과 원근 투영

[G3b] 10~14쪽 | 아래 행렬은 기본 OpenGL 관례의 보충

비교	직각/직교 투영	원근 투영
보이는 공간	직육면체	절두체(잘린 피라미드)
물체 크기	거리가 달라도 같은 크기	멀수록 작게 보임
용도	도면·UI·등각적 표현 등	사람 눈과 유사한 게임 카메라
함수	glm::ortho(l,r,b,t,n,f)	glm::perspective(fovy,aspect,n,f)

투영 행렬의 직접 출력은 2D 픽셀 좌표가 아니라 **클립 좌표**입니다. 그 뒤 클리핑과 원근 나누기를 거칩니다. 원근 투영의 near·far는 카메라에서의 양수 거리이며 0<n<f입니다.

직각 투영: 경계를 -1~1에 대응

xndc=2xeye/(r-l)-(r+l)/(r-l)
yndc=2yeye/(t-b)-(t+b)/(t-b)
zndc=-2zeye/(f-n)-(f+n)/(f-n)

오른손 뷰 공간에서 가까운 평면은 zeye=-n, 먼 평면은 -f입니다. 직각 투영은 강의처럼 n이 음수인 설정도 가능하므로 원근 투영의 양수 조건을 그대로 일반화하지 않습니다.

원근 투영: 거리로 나누어 작아진다

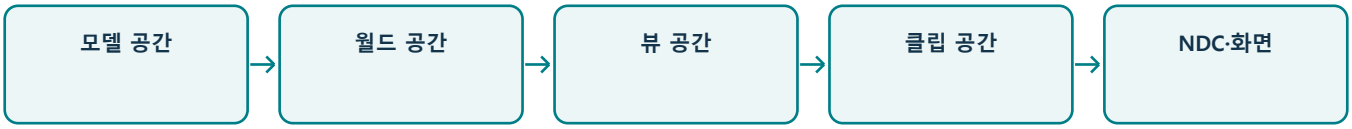
c=1/tan(fovy/2), a=width/height
xclip=(c/a)xeye, yclip=c yeye, wclip=-zeye
zclip=-[(f+n)/(f-n)]zeye-2fn/(f-n)
xndc=xclip/wclip, yndc=yclip/wclip

보충 예: fovy=90도, aspect=1에서 c=1. (1,0,-2)는 xndc=0.5, (1,0,-4)는 0.25입니다. 거리가 두 배면 화면 중심에서의 변위가 절반입니다.

시야각이 커지면 더 넓게 보여 같은 물체가 작아집니다. 창 비율이 바뀌면 viewport와 필요한 projection aspect를 함께 갱신합니다. near를 0으로 두지 않습니다. 행렬식은 GLM의 좌표계·깊이 범위 설정을 변경하지 않은 기본 관례입니다.

29 MVP부터 화면 좌표까지

[G3a] 14~16쪽; [G3b] 2쪽, 12~19쪽



vclip=Projection × View × Model × vlocal
클리핑: -wclip≤xclip,yclip,zclip≤wclip
원근 나누기: vndc=(xclip,yclip,zclip)/wclip

기본 OpenGL 관례에서 가시 영역의 NDC는 각 축 -1~1입니다. **모델 좌표가 반드시 -1~1이어야 하는 것은 아닙니다.** 큰 모델 좌표도 변환 후 가시 범위에 들어오면 보일 수 있습니다.

xwin=x0+(xndc+1)width/2
ywin=y0+(yndc+1)height/2
기본 깊이 범위: zwin=(zndc+1)/2

glViewport(x0,y0,width,height)의 원점은 출력 영역 왼쪽 아래입니다. 픽셀 중심은 일반적으로 반정수 좌표이므로 NDC=1의 경계가 곧 마지막 픽셀의 중심은 아닙니다.

보충 종합 예제

이미 MVP를 적용한 클립 좌표가 (2,-1,0,4), viewport가 (100,50,800,600)이라고 합시다. 세 좌표 모두 [-4,4] 안이므로 클립 범위를 통과합니다.

단계	계산	결과
NDC	(2/4,-1/4,0/4)	(0.5,-0.25,0)
화면 x	100+(0.5+1)×800/2	700
화면 y	50+(-0.25+1)×600/2	275
깊이	(0+1)/2	0.5

마우스 좌표와의 연결: 보충

GLFW 커서 좌표는 콘텐츠 영역의 왼쪽 위 기준으로 아래쪽 y가 증가합니다. 논리적 창 전체를 NDC에 대응시키면 xndc=2x/W-1, yndc=1-2y/H입니다. HiDPI, 부분 viewport, 실제 픽셀 선택에서는 좌표 스케일과 영역의 시작점을 추가로 반영해야 합니다.

30 깊이 검사·컬링·모델 데이터

[G3a] 28~41쪽

기능	깊이 테스트	면 컬링
대상	프래그먼트의 깊이	삼각형 등 프리미티브의 앞뒷면
목적	겹친 물체 중 앞의 표면 표시	설정한 방향의 면을 아예 제외
설정	glEnable(GL_DEPTH_TEST)	glEnable(GL_CULL_FACE)
관련 상태	깊이 버퍼·깊이 비교 함수	glFrontFace, glCullFace

기본 깊이 비교 GL_LESS에서는 저장값보다 작은 깊이가 통과합니다. 보충 예: 깊이 0.7을 기록한 뒤 같은 픽셀의 0.3은 통과하고 0.8은 탈락합니다. 프레임마다 깊이 버퍼도 적절히 초기화해야 이전 프레임의 깊이가 남지 않습니다.

기본 앞면은 화면에서 반시계 **CCW**, 기본 컬링 대상은 뒤쪽 BACK입니다. 컬링 기능 자체는 별도로 켜야 합니다. 음수 스케일 등으로 면 방향이 뒤집힐 수 있습니다. 깊이 검사와 컬링은 서로 대체하지 않습니다.

GLU quadric 객체

함수	뜻
gluNewQuadric / gluDeleteQuadric	quadric 설명 객체 생성 / 정리
gluQuadricDrawStyle	FILL·LINE·SILHOUETTE·POINT
gluQuadricNormals / Orientation	NONE·FLAT·SMOOTH / OUTSIDE·INSIDE
gluSphere	반지름, slices, stacks로 구 분할
gluCylinder	아래·위 반지름, 높이, 분할 수. 위 반지름 0이면 원뿔 형태
gluDisk	안·밖 반지름과 분할 수로 원판/고리

이 GLU 도형 경로는 전통적인 OpenGL 사용과 연결됩니다. 3.3 Core의 VAO/VBO 셰이더 경로와 그대로 호환된다고 가정하지 않습니다.

OBJ와 MTL

v: 정점, **vt**: 텍스처 좌표, **vn**: 법선, **f**: 면. f 1/2/3은 위치1·텍스처2·법선3의 조합이며 f 1//3은 텍스처 생략입니다. 양수 인덱스는 1부터 시작합니다. MTL은 재질·텍스처 관련 정보를 담습니다.

읽기 순서: 정점·면 개수 집계 → 저장 공간 준비 → 데이터 읽기 → 필요한 삼각형과 속성 구성. 원본의 단순 sscanf 예제는 v와 단순 삼각형 f만 처리하므로 / 구분자, 다각형, 상대 인덱스 등을 자동 지원하지 않습니다.

31 시험 직전 공식 모음 I

선·원·2D 변환·클리핑 | 각 공식의 적용 조건을 함께 기억

항목	핵심 식과 조건
기울기	m=Δy/Δx. Δx=0은 수직선 별도.
DDA	steps=max(|Δx|,|Δy|), 증분=(Δx/steps,Δy/steps). 누적은 실수, 출력만 반올림.
브레즌햄 선	0≤m≤1, x 증가: p1=2Δy-Δx. p<0이면 E, p+=2Δy. 그 외 NE, p+=2(Δy-Δx).
브레즌햄 원	중심 상대 (0,r), p1=3-2r. p<0: p+=4x+6, E. 그 외 p+=4(x-y)+10, SE. 갱신은 이전 x,y.
이동·신축	(x′,y′)=(x+tx,y+ty), (sx x,sy y).
2D 회전	x′=x cosθ-y sinθ, y′=x sinθ+y cosθ.
합성·기준점	먼저 A 다음 B → BA. 중심 q 회전 → T(q)R T(-q).
반사	x축:(x,-y), y축:(-x,y), y=x:(y,x), y=-x:(-y,-x).
밀림	x′=x+hx y 또는 y′=y+hy x. 다른 성분은 유지.
윈도우 매핑	sx=뷰포트폭/윈도우폭, xv=xvL+(xw-xwL)sx. y도 같은 원리.
영역코드	왼쪽1, 오른쪽2, 아래4, 위8. OR=0 수락, AND≠0 거부.
Liang-Barsky	pk u≤qk. p<0이면 하한 max, p>0이면 상한 min. p=0,q<0은 거부.
보간	(1-t)A+tB. t는 시작에서 끝으로 간 비율.

검산 체크

- 선의 마지막 출력이 끝점과 맞는가? 원의 대칭점에 중심을 더했는가?
- 임의 중심을 직접 대입했을 때 중심이 고정되는가? 변환 순서를 바꾸지 않았는가?
- 클리핑 후 끝점이 모든 경계 조건을 만족하는가? u는 [0,1] 안인가?

32 시험 직전 공식 모음 II

조명·GPU 데이터·카메라·투영 | 보충식 포함

항목	핵심 식과 조건
정규화·내적	normalize(a)=a/|a|, a≠0. 단위벡터이면 a·b=cosθ.
외적	a×b=(ay bz-az by, az bx-ax bz, ax by-ay bx). 순서를 바꾸면 부호 반대.
반사 방향	R=2(N·L)N-L. L은 표면→광원.
Phong 밝기	I=Ka Ia+(Ip/d){Kd max(0,N·L)+Ks[max(0,V·R)]^n}. 강의의 거리 계수 유지.
Gouraud / Phong	밝기 보간 / 법선 보간 후 정규화·조명 계산.
시간	Δt=t현재-t이전, 이동량=속도×Δt.
정점 저장량	정점 수×정점당 byte. EBO는 인덱스 수×인덱스당 byte.
속성 주소	offset+정점 번호×stride. stride·offset 모두 byte.
동차 좌표	점 w=1, 방향 w=0. 일반 점 w≠0 → (x/w,y/w,z/w).
GLM 합성	translate 다음 rotate 다음 scale → M=TRS → 실제 적용 S,R,T.
MVP	vclip=Projection View Model vlocal.
카메라	f=normalize(At-Eye), r=normalize(f×Up), u=r×f.
원근 축소	c=1/tan(fovy/2), xndc=(c/aspect)xeye/(-zeye), yndc=c yeye/(-zeye).
화면 좌표	xwin=x0+(xndc+1)w/2, ywin=y0+(yndc+1)h/2.

부호·단위 다섯 가지: 회전은 반시계 양수, GLM 각은 라디안, 카메라 전방은 -z, OpenGL viewport는 왼쪽 아래 기준, 광원 L은 표면→광원입니다.

33 자가 점검 문제

풀이를 가리고 먼저 계산하세요. 해설은 다음 절입니다. 보충 문제.

번호	문제
1	1024×768 화면, RGBA 각 8bit 컬러 버퍼 두 개의 저장량을 byte와 MiB로 구하세요. 다른 버퍼·정렬은 제외합니다.
2	DDA로 (0,0)~(4,2)를 그릴 때 누적 실수 좌표와 출력 픽셀을 쓰세요. 양수 .5는 올립니다.
3	브레즌햄으로 (0,0)~(5,3)의 p1, 첫 3번 선택과 다음 p를 구하세요.
4	중심 (2,-1), 반지름 5인 원에서 상대좌표 (3,4)의 8방향 대칭점을 실제 좌표로 바꾸세요.
5	(3,2)를 (1,1) 중심으로 반시계 90도 회전한 다음, 원점 기준 2배 신축하세요. 합성식을 쓰세요.
6	윈도우 (0,0)~(20,10), 뷰포트 (100,50)~(500,250)에서 (5,8)의 화면 좌표와 모양 왜곡 여부를 구하세요.
7	윈도우 [0,10]×[0,10]에 대해 (-2,4),(12,4)의 영역코드와 최종 선분을 구하세요. 두 끝점 AND=0의 의미도 설명하세요.
8	Liang-Barsky로 같은 선분의 유효 u 구간을 구하세요.
9	Ia=0.2,Ka=0.5,Ip=4,d=2,Kd=0.5,Ks=0.25,n=2, N·L=0.6,V·R=0.8일 때 총 밝기를 구하세요.
10	위치 vec3·법선 vec3·UV vec2를 float로 번갈아 저장합니다. stride, 법선 offset, UV offset, 정점 100개의 VBO 크기를 구하세요.
11	Eye=(0,0,3), At=(0,0,0), Up=(0,1,0)인 뷰에서 월드 점 (2,1,-1)의 뷰 좌표를 구하세요.
12	클립 좌표 (1,-1,0,2), viewport (0,0,800,600)의 NDC와 화면 좌표를 구하세요.
13	Gouraud와 Phong 쉐이딩에서 보간하는 값은 무엇인가요? Phong 조명 모델과의 차이도 설명하세요.
14	M=translate(I,(1,0,0)), 이후 M=rotate(M,90도,z축). (1,0,0,1)에 적용한 결과를 구하세요.

34 자가 점검 해설

핵심 계산과 틀리기 쉬운 이유

1~4 저장량과 픽셀

- **1.** 1024×768×4×2=6,291,456byte=**6MiB**. bit를 byte로 바꾸었는지 확인합니다.
- **2.** steps=4, 증분=(1,0.5). 실수 좌표 (0,0),(1,0.5),(2,1),(3,1.5),(4,2). 출력 (0,0),(1,1),(2,1),(3,2),(4,2).
- **3.** Δx=5,Δy=3,p1=1. 첫 선택 NE→(1,1), 다음 p=-3. 둘째 E→(2,1), 다음 p=3. 셋째 NE→(3,2), 다음 p=-1.
- **4.** 상대 (±3,±4),(±4,±3)에 (2,-1)을 더합니다. 결과는 (5,3),(-1,3),(5,-5),(-1,-5),(6,2),(-2,2),(6,-4),(-2,-4).

5~8 변환과 클리핑

- **5.** M=S(2,2)T(1,1)R(90도)T(-1,-1). (3,2)→(2,1)→(-1,2)→(0,3)→**(0,6)**.
- **6.** sx=400/20=20,sy=200/10=20. (5,8)→**(200,210)**. 균등 배율이므로 가로세로 비율은 유지됩니다.
- **7.** 코드는 0001,0010. AND=0000은 즉시 거부할 수 없다는 뜻이며, 이것만으로 모든 선분이 교차한다고 단정할 수는 없습니다. 이 예제는 수평선이므로 남는 선분은 **(0,4)~(10,4)**.
- **8.** dx=14,dy=0. 왼쪽 u≥2/14=1/7, 오른쪽 u≤12/14=6/7. 수평 경계는 p=0,q≥0이므로 제한 없음. **u∈[1/7,6/7]**.

9~14 밝기·버퍼·카메라

- **9.** 주변=0.1, 산란=(4/2)×0.5×0.6=0.6, 거울=(4/2)×0.25×0.8²=0.32. 합계 **1.02**. 클램프 조건이 없으므로 원값입니다.
- **10.** 8float이므로 stride=**32byte**, 법선 offset=12, UV offset=24, VBO=100×32=**3200byte**.
- **11.** 이 카메라의 회전 축은 기본 축과 같아 z에서 3을 뺍니다. 뷰 좌표 **(2,1,-4)**.
- **12.** NDC=(0.5,-0.5,0). 화면=(600,150), 기본 깊이=0.5.
- **13.** Gouraud는 정점의 밝기/색, Phong 쉐이딩은 법선을 보간하고 각 위치에서 조명 계산. Phong 조명 모델은 주변·산란·거울반사의 한 점 밝기 식입니다.
- **14.** M=TR이므로 회전 먼저: (1,0,0)→(0,1,0)→**(1,1,0)**. w=1은 유지됩니다.

35 원본 정정·범위와 출처

교수 자료를 따르되 계산 오류는 구분합니다.

위치	시험 대비 주의점
[T2] 원 알고리즘	p 갱신식에 이동 전의 중심 상대좌표를 사용합니다. 절대좌표를 그대로 넣지 않습니다.
[T2] 주사선 채우기	끝점 중복 처리 규칙이 완전하게 명시되지 않았습니다. 강의 예제 재현과 표준 반열린 구간 구현을 구분합니다.
[T3] 17쪽	유사 변환의 신축은 균등 신축으로 제한해야 모양이 유지됩니다.
[T4] 14~17쪽	경계 포함 부등식은 pk u≤qk. 평행한 경우 qk≥0이면 그 경계에서 추가 제한 없음. 모든 경계의 하한·상한을 누적합니다.
[T4] 24쪽	Weiler-Atherton의 외부/내부 추적 설명은 축약되어 있습니다. 진입·이탈 교점에서 필요한 경계를 바꾸는 원리로 이해합니다.
[T5] 4,9~12쪽	물리적 거리 제곱 반비례 설명과 계산식 Ip/d가 공존합니다. 이 문서의 조명 수치는 강의 계산식 Ip/d를 사용했습니다.
[G2a] 10,12쪽	비트 연산 허용 설명이 모순됩니다. GLSL 3.30의 정수 비트 연산은 지원됩니다. vec4/vec3, 변수명 등 예제 오타는 그대로 외우지 않습니다.
[G3a] 초기화·행렬	단위행렬은 glm::mat4(1.0f)로 명시합니다. 기본 생성자의 값을 무조건 0이나 I로 가정하지 않습니다.
[G3b] 5쪽	R의 세 번째 행이 -f이므로 View의 마지막 열 세 번째 값은 +f·Eye입니다. Eye를 대입해 원점인지 검산합니다.

원본 자료 목록: 총 323쪽

[T1] 1. 컴퓨터 그래픽스 개요 시스템.pdf · 19쪽

[T2] 2. 2차원 그래픽스의 기본요소.pdf · 35쪽

[T3] 3. 2차원 그래픽스의 변환.pdf · 19쪽

[T4] 4. 2차원 그래픽스의 윈도우와 뷰포트.pdf · 25쪽

[T5] 5. 조명모델.pdf · 20쪽

[G1] 지엘1 오픈지엘 기초.pdf · 60쪽

[G2a] 지엘2 GLSL 기초 1.pdf · 38쪽

[G2b] 지엘2 GLSL 기초 2.pdf · 47쪽

[G3a] 지엘3 좌표계 변환 1.pdf · 41쪽

[G3b] 지엘3 좌표계 변환 2.pdf · 19쪽

전체 텍스트를 검토하고 주요 공식·도식을 원본 렌더링과 대조했습니다. 보충 설명은 강의의 계산을 이해하기 위한 것으로, 추가 기출이나 교수님의 실제 출제 비중을 뜻하지 않습니다.

F01 함수 문제를 읽는 순서

함수 중심 보강 | [G3a] 2~5쪽, 9~13쪽 | 수치 예제는 보충

이 보강편은 **함수의 이름과 인자가 무엇을 의미하는지 해석하고, 코드로부터 최종 결과를 계산**하는 데 집중합니다. 실제 출제 비중은 확인되지 않았지만, 사용자가 설명한 시험 유형에 맞춰 구성했습니다.

함수 호출은 네 가지를 구분해서 읽기

질문	예: M=glm::translate(M,glm::vec3(2,3,0))
무슨 역할인가?	이동 변환을 현재 행렬에 결합한 새 행렬을 만듭니다.
인자가 무엇인가?	첫 M은 기존 행렬, vec3는 x·y·z 방향 이동량입니다.
무엇을 반환하는가?	기존 M의 오른쪽에 이동행렬 T를 곱한 M×T를 반환합니다.
점은 언제 변하는가?	p2=M*p처럼 행렬을 좌표에 적용할 때입니다. 함수만 호출하면 점 배열이 자동 변경되지는 않습니다.

기존 행렬 Mold → 새 행렬 Mnew=Mold×T
최종 점 p'=Mnew×p

기본 준비 함수

표현	의미	보충 계산
glm::mat4(1.0f)	주대각선이 1인 4×4 단위행렬 I	I*p=p: 점을 그대로 유지
glm::vec3(x,y,z)	3개 성분을 가진 벡터/좌표 값 생성	vec3(2,3,4)는 세 값 (2,3,4)
glm::vec4(v,1.0f)	3성분 v를 점의 동차 좌표로 확장	(x,y,z,1)
glm::radians(degree)	도 단위 각을 라디안으로 변환	90도→π/2, 180도→π
glm::degrees(radian)	라디안을 도 단위로 변환	π/2→90도

꼭 기억: mat4(2.0f)는 대각선 전체가 2인 행렬입니다. 3차원 물체를 2배 키우는 표준 신축행렬과는 마지막 w 성분이 다릅니다. 물체 크기 변환은 glm::scale로 표현하세요.

GLM의 변환 함수는 보통 **새 행렬을 반환**하므로 M=...처럼 결과를 받아야 합니다. 이미 있는 M을 초기화할지 누적할지도 문제에서 확인합니다.

F02 glm::translate: 옮기는 함수

[G3a] 9~10쪽, 19~22쪽 | 보충 코드 계산

역할과 인자

glm::translate(M, glm::vec3(tx, ty, tz))

• **M**: 이미 만들어 둔 변환행렬. **tx,ty,tz**: 각 축 방향 이동량. 양수는 해당 축의 양의 방향, 음수는 반대 방향입니다.

• **반환**: M×T. M이 단위행렬일 때는 이동행렬 T만 얻습니다. 이동은 물체의 크기나 각 점 사이의 거리를 바꾸지 않습니다.

```
glm::mat4 M(1.0f);
M = glm::translate(M, glm::vec3(3.0f, -2.0f, 1.0f));
glm::vec4 p(2.0f, 5.0f, -1.0f, 1.0f);
glm::vec4 q = M * p;
```

x′=2+3=5, y′=5-2=3, z′=-1+1=0
q=(5,3,0,1)

방향벡터라면 왜 이동하지 않을까?

일반 동차 벡터: T(x,y,z,w)=(x+tx w,y+ty w,z+tz w,w)
점 w=1: 이동량이 더해짐
방향 w=0: 이동량이 더해지지 않음

같은 코드에서 p의 마지막 성분이 0이라면 q=(2,5,-1,0)입니다. 방향벡터는 위치가 없으므로 평행이동의 영향을 받지 않습니다.

연속 이동과 흔한 실수

```
M = glm::translate(M, glm::vec3(2, 0, 0));
M = glm::translate(M, glm::vec3(0, 3, 0));
```

위 두 줄을 I에서 시작하면 T(2,0,0)T(0,3,0)=T(2,3,0)입니다. 하지만 기존 M에 회전이나 신축이 있으면 인자로 넣은 이동량이 최종 월드에서 그대로 보인다고 단정할 수 없습니다. 예: M=S(2)인 상태에서 translate(M,(1,0,0))이면 ST이므로 원점은 최종 (2,0,0)으로 갑니다.

답안 쓰기: "x로 3, y로 -2, z로 1 이동하는 행렬을 만들고, 점 (2,5,-1)에 적용하므로 (5,3,0)이다." 기능 설명과 계산을 함께 적습니다.

F03 glm::scale: 크기를 바꾸는 함수

[G3a] 11쪽, 19쪽, 23~24쪽 | 보충 코드 계산

glm::scale(M, glm::vec3(sx, sy, sz))

- **sx,sy,sz**: 각 축의 배율. 거리를 더하는 양이 아닙니다. 반환값은 M×S입니다.
- M=I이면 x′=sx x, y′=sy y, z′=sz z입니다. **원점 기준**이므로 물체 크기뿐 아니라 원점에서 떨어진 위치도 바뀝니다.

배율	해석
2	해당 축 좌표 2배
0.5	해당 축 좌표 절반
1	해당 축 변화 없음
-1	해당 축 방향 반전
0	해당 축의 모든 좌표가 0으로 모여 차원이 줄어듦

glm::mat4 M(1.0f);

M = glm::scale(M, glm::vec3(2.0f, 0.5f, -1.0f));

glm::vec4 q = M * glm::vec4(3, 4, 2, 1);

q=(2×3,0.5×4,-1×2,1)=(6,2,-2,1)

삼각형에 적용하면

A=(1,1),B=(3,1),C=(1,2)를 x 2배, y 3배 신축하면 A′=(2,3),B′=(6,3),C′=(2,6)입니다. AB의 길이는 2→4, AC는 1→3이므로 모양의 비율도 달라집니다.

균등 신축은 모든 축의 배율이 같아 모양이 유지됩니다. **비균등 신축**은 서로 달라 일반적으로 각도와 모양이 변합니다. z를 유지할 2D 신축이면 sz=1을 둡니다.

이미 옮긴 물체를 키우는 것처럼 보이는 코드

M = glm::translate(glm::mat4(1.0f), glm::vec3(5, 0, 0));

M = glm::scale(M, glm::vec3(2, 2, 2));

M=TS입니다. 정점에는 신축 후 이동이 적용되므로 모델 원점은 여전히 (5,0,0)입니다. 반대로 M=ST이면 원점 이동량까지 2배가 되어 (10,0,0)입니다. 함수 이름만 보고 "이동 후 신축"이라고 읽지 않습니다.

F04 glm::rotate: 축 주위로 돌리는 함수

[G3a] 12쪽, 19쪽, 25~27쪽 | 보충 축별 예제

glm::rotate(M, angle, glm::vec3(ax, ay, az))

• **M**: 기존 행렬. **angle**: 라디안 회전각. **axis**: 회전축의 방향입니다. vec3의 세 성분은 축 방향이지 x/y/z 각각의 회전각이 아닙니다.

• 반환값은 **M×R**. 축은 0벡터를 사용하지 않으며 방향을 단위화해 이해합니다. GLM의 해당 함수는 주어진 비영 축을 내부적으로 정규화합니다.

axis	회전축	양의 90도 보충 예
(1,0,0)	x축	(0,1,0)→(0,0,1)
(0,1,0)	y축	(1,0,0)→(0,0,-1)
(0,0,1)	z축	(1,0,0)→(0,1,0)

오른손 좌표계의 양의 각도는 오른손 엄지를 축의 양의 방향으로 두었을 때 나머지 손가락이 감기는 방향입니다. **축의 양의 쪽에서 원점을 바라보면 반시계**입니다.

glm::mat4 M(1.0f);

M = glm::rotate(M, glm::radians(90.0f), glm::vec3(0,0,1));

glm::vec4 q = M * glm::vec4(2,1,3,1);

z축: x′=x cosθ-y sinθ, y′=x sinθ+y cosθ, z′=z
θ=90도: (x,y,z)→(-y,x,z)
q=(-1,2,3,1)

각도와 축의 함정

• angle=90.0f를 그대로 넣으면 **90라디안**입니다. 90도와 다릅니다. glm::radians(90.0f)로 변환합니다.

• axis=(0,0,2)는 z축 방향입니다. 2배 각도로 돈다는 뜻이 아닙니다. axis=(0,0,-1)은 반대 방향의 축이므로 양의 각도가 +z축의 음의 각도에 대응합니다.

• 이 함수의 기본 회전축은 모델/행렬 조합의 **원점을 지나는 축**입니다. 임의 위치의 축 주위로 돌리려면 기준점을 옮기는 합성이 필요합니다.

F05 여러 함수를 보고 적용 순서 계산

[G3a] 20쪽, 25~27쪽 | 보충 추적 예제

GLM 누적: M=translate(M,t) → Mnew=Mold T
M=rotate(M,θ,a) → Mnew=Mold R
M=scale(M,s) → Mnew=Mold S
열벡터 p에는 오른쪽부터 적용

코드: I에서 시작	최종 행렬	점 (1,0)의 결과
translate(2,0) 다음 rotate(90도,z)	TR	회전 (0,1) → 이동 (2,1)
rotate(90도,z) 다음 translate(2,0)	RT	이동 (3,0) → 회전 (0,3)

세 함수의 종합 계산

```
glm::mat4 M(1.0f);
M = glm::translate(M, glm::vec3(3,1,0));
M = glm::rotate(M, glm::radians(90.0f), glm::vec3(0,0,1));
M = glm::scale(M, glm::vec3(2,3,1));
glm::vec4 q = M * glm::vec4(1,2,0,1);
```

실제 적용	좌표	설명
초기	(1,2,0)	모델의 점
S	(2,6,0)	x 2배, y 3배
R	(-6,2,0)	z축 90도: (x,y)→(-y,x)
T	(-3,3,0)	x+3, y+1

같은 변수명 M이지만 계산 단계는 다르다

첫 줄 다음 M=T, 둘째 줄 다음 M=TR, 셋째 줄 다음 M=TRS입니다. 반환된 행렬을 M에 다시 저장하는 것입니다. 수학 문제에서 "먼저 이동하고, 그 다음 회전한 점"을 요구하면 RTp로 써야 하며, 이를 GLM 누적 코드로 만들 때는 rotate 다음 translate 순서로 호출합니다.

검산: 모델 원점 (0,0,0)에 TRS를 적용하면 (3,1,0)입니다. 이 코드에서 마지막 이동량이 보존되는지 확인할 수 있습니다. 행렬을 매 프레임 I로 되돌리는지, 이전 프레임의 M에 계속 누적하는지도 구분합니다.

F06 기준점을 정해 회전·신축하기

[T3] 6,8,11쪽; [G3a] 합성 원리 | 보충 코드

임의 중심 회전의 기능

q=(2,1,0)을 중심으로 도형을 돌리고 싶다면 q를 원점으로 옮기고 회전한 뒤 되돌립니다. "물체 중심에서 회전"과 "월드 원점 주위를 공전"은 결과가 다릅니다.

```
glm::vec3 center(2,1,0);
glm::mat4 M(1.0f);
M = glm::translate(M, center);
M = glm::rotate(M, glm::radians(90.0f), glm::vec3(0,0,1));
M = glm::translate(M, -center);
```

M=T(center) R T(-center)
실제 점 적용: 중심을 빼기 → 회전 → 중심을 더하기

점 p=(4,2,0)	과정
중심 빼기	(4,2,0)-(2,1,0)=(2,1,0)
90도 회전	(2,1,0)→(-1,2,0)
중심 더하기	(-1,2,0)+(2,1,0)=(1,3,0)

중심 (2,1,0) 자체를 넣으면 (0,0,0)→(0,0,0)→(2,1,0)으로 돌아옵니다. **기준점이 고정되는지**가 가장 간단한 검산입니다.

임의 중심 신축

```
glm::mat4 S(1.0f);
S = glm::translate(S, center);
S = glm::scale(S, glm::vec3(2,2,1));
S = glm::translate(S, -center);
```

같은 p=(4,2,0)은 상대좌표 (2,1,0)을 두 배 한 (4,2,0)에 중심을 더해 **(6,3,0)**이 됩니다. 물체의 모델 원점이 이미 회전 중심이라면 이 왕복 이동 없이 R만으로 자전할 수 있습니다.

자전과 공전을 코드로 구분

TRp: 원점 기준으로 회전한 모양을 t에 배치합니다. **RTp**: 먼저 t만큼 떨어뜨린 뒤 월드 원점 주위로 회전시킵니다. 단, 실제 "물체 중심"은 정점 배치와 모델 원점 정의에 따라 달라집니다.

F07 glm::lookAt: 카메라 기준으로 바꾸기

[G3b] 3~9쪽 | 보충 수치 계산

glm::lookAt(eye, target, up)

인자	의미	주의
eye	월드 공간 카메라 위치	카메라의 좌표
target / At	카메라가 바라보는 월드 공간의 점	방향벡터 자체가 아님
up	카메라 위쪽을 정하는 기준 방향	target-eye와 평행하면 축 생성 불가

이 함수는 카메라를 직접 화면에 그리는 함수가 아니라 **월드 좌표를 카메라 기준 좌표로 바꾸는 View 행렬**을 반환합니다. 그 행렬을 정점에 곱해야 효과가 나타납니다.

```
glm::mat4 V = glm::lookAt(
    glm::vec3(0,0,5), glm::vec3(0,0,0), glm::vec3(0,1,0));
glm::vec4 q = V * glm::vec4(2,1,1,1);
```

이 경우 축은 기본 축과 같고 카메라 위치만 z=5
뷰 좌표=(x,y,z-5)
(2,1,1) → (2,1,-4)

카메라 앞쪽은 뷰 공간의 -z입니다. 따라서 z=-4는 "카메라보다 앞쪽으로 4 떨어졌다"는 의미입니다. 월드에서의 z=1과 카메라 기준의 z=-4는 서로 다른 좌표입니다.

회전된 카메라도 계산하는 일반식

f=normalize(target-eye), r=normalize(f×up), u=r×f
a=worldPoint-eye
viewPoint=(r·a, u·a, -f·a)

보충 예: eye=(5,0,0), target=(0,0,0), up=(0,1,0). f=(-1,0,0), r=(0,0,-1), u=(0,1,0). 월드 점 (1,2,3)은 a=(-4,2,3)이므로 **(-3,2,-4)**로 바뀝니다.

확인: eye를 넣으면 항상 뷰 원점 (0,0,0)입니다. 카메라 위치를 물체 좌표에 단순히 더하는 것은 일반적으로 잘못된 뷰 계산입니다. 회전된 카메라에서는 위치를 뺀 후 축에 대한 내적까지 필요합니다.

F08 glm::ortho·perspective: 투영 함수

[G3b] 10~14쪽 | 좌표 계산은 기본 OpenGL 관례의 보충

함수	인자	역할
glm::ortho(l,r,b,t,n,f)	좌·우·아래·위 경계, near, far	거리에 따른 크기 축소가 없는 직각 투영
glm::perspective(fovy,aspect,n,f)	수직 시야각, 가로/세로 비, near, far	멀수록 작게 보이는 원근 투영

두 함수 모두 **투영행렬**을 반환합니다. 기존 M을 받는 translate/rotate/scale과 달리, 첫 인자가 기존 행렬이 아닙니다. 점에 P를 곱하면 먼저 클립 좌표가 나오고 이후 w로 나누어 NDC를 얻습니다.

직각 투영 예제

glm::mat4 P = glm::ortho(-10.0f,10.0f,-5.0f,5.0f,1.0f,11.0f);

xndc=2x/20=x/10, yndc=2y/10=y/5
zndc=-2z/10-12/10
뷰 점 (5,2.5,-6) → NDC (0.5,0.5,0)

z는 뷰 좌표입니다. 가까운 평면은 z=-1, 먼 평면은 z=-11로, 각각 NDC -1,+1에 대응합니다. 이 예제의 x,y는 거리 z가 바뀌어도 같은 값입니다.

원근 투영 예제

glm::mat4 P = glm::perspective(glm::radians(90.0f),2.0f,1.0f,11.0f);

c=1/tan(90도/2)=1, aspect=2
xclip=0.5x, yclip=y, wclip=-z
뷰 점 (2,1,-4) → 클립 (1,1,2.6,4)
NDC=(0.25,0.25,0.65)

같은 크기의 물체를 두 배 멀리 놓으면 x,y의 화면 변위가 절반이 됩니다. fovy가 커지면 더 넓게 보이고 물체는 작아집니다. aspect는 width/height이며 정수 나눗셈을 피해야 합니다.

조건: 원근 투영은 라디안 시야각, 0<near<far. 위 예제는 GLM의 오른손 좌표계·NDC 깊이 -1~1 기본 설정입니다. 함수가 알아서 Model·View까지 적용하는 것은 아닙니다.

F09 glViewport: 화면의 어디에 그릴까?

[G1] 51쪽; [G3b] 15~19쪽 | 화면 좌표 계산은 보충

glViewport(x0, y0, width, height);

• **x0,y0**: 프레임 버퍼의 왼쪽 아래를 기준으로 뷰포트 시작 위치. **width,height**: 영역의 픽셀 크기입니다. 오른쪽 위 좌표가 아닙니다.

• 반환값은 없으며, OpenGL의 **뷰포트 상태**를 설정합니다. GLM처럼 행렬을 반환하는 함수가 아닙니다. NDC가 이 영역에 대응되도록 합니다.

xwin=x0+(xndc+1)width/2
ywin=y0+(yndc+1)height/2

glViewport(100,50,400,200);

NDC (x,y)	계산한 화면 좌표	의미
(-1,-1)	(100,50)	왼쪽 아래 경계
(0,0)	(300,150)	뷰포트 중앙
(1,1)	(500,250)	오른쪽 위 경계
(0.5,-0.5)	(400,100)	중앙보다 오른쪽·아래

경계 좌표 (500,250)를 마지막 픽셀의 중심으로 해석하지 않습니다. width=400은 시작 x에 더해질 너비입니다. 실제 픽셀 중심은 일반적으로 반정수 좌표입니다.

Projection과 Viewport를 구분

바꾸는 것	효과
perspective의 fovy	카메라가 보는 각도·넓이와 원근 크기 관계
perspective의 aspect	투영 시 가로세로 비율
glViewport의 width/height	NDC가 실제 화면에서 차지할 위치·크기
lookAt의 eye/target	장면을 바라보는 위치·방향

창이 바뀌면 glViewport를 새 프레임 버퍼 크기로 설정하고 원근 투영의 aspect도 필요하면 갱신합니다. viewport만 바꾸고 aspect를 유지하면 물체가 가로로 늘어나는 등 왜곡될 수 있습니다. glViewport는 임의 사각 영역의 모든 쓰기를 제한하는 scissor 기능과는 다릅니다.

F10 벡터·보간 함수도 계산 문제로 읽기

[G3a] 4~5쪽; [G2a] 19쪽 | 보충 예제

함수	기능	직접 계산 예
glm::length(v)	벡터 길이	v=(3,4,0) → √(9+16)=5
glm::normalize(v)	길이 1, 방향 유지	(3,4,0) → (0.6,0.8,0)
glm::distance(a,b)	두 점 사이 거리	a=(1,2,0),b=(4,6,0) → 5
glm::dot(a,b)	성분별 곱의 합, 내적	(1,2,3)·(4,0,-1)=1
glm::cross(a,b)	두 벡터에 수직인 외적	(1,0,0)×(0,1,0)=(0,0,1)
glm::clamp(x,lo,hi)	범위 밖을 경계값으로 제한	clamp(1.3,0,1)=1
glm::mix(a,b,t)	두 값 사이 비율 보간	mix(2,10,0.25)=4
glm::sin / cos	라디안 삼각함수	sin(π/2)=1, cos(π)=-1

표의 숫자는 계산 의미를 설명하기 위한 값입니다. 실제 C++ 호출에서는 함수가 요구하는 타입을 맞춥니다. 예: glm::clamp(1.3f,0.0f,1.0f). GLSL에는 glm:: 없이 같은 이름의 내장 함수가 있습니다.

법선과 광원 방향을 함수로 구하는 예

```
glm::vec3 position(0,0,0);
glm::vec3 lightPos(0,3,4);
glm::vec3 N(0,0,1);
glm::vec3 L = glm::normalize(lightPos-position);
float diffuse = glm::max(glm::dot(N,L),0.0f);
```

lightPos-position=(0,3,4), 길이=5
L=(0,0.6,0.8), dot(N,L)=0.8
diffuse=0.8

normalize를 빼면 dot 결과가 4가 되어 코사인 값으로 해석할 수 없습니다. 0벡터는 정규화할 수 없습니다. cross의 인자를 바꾸면 결과 부호가 반대입니다.

정규화라는 말이 같아도 기능은 다르다

normalize(v)는 벡터 길이를 1로 만드는 수학 함수입니다. glVertexAttribPointer의 normalized는 정수 속성을 실수 범위로 바꾸는 데이터 해석 옵션이므로 같은 작업이 아닙니다. normalize가 화면 좌표를 NDC로 변환해 주는 것도 아닙니다.

F11 행렬이 실제 화면에 쓰이는 연결

[G2a] 16~18,35~38쪽; [G2b] 25,35~46쪽; [G3b] 16~18쪽

CPU에서 만들고, GPU에 보내고, 정점에 적용

```
glm::mat4 model(1.0f);
model = glm::translate(model, glm::vec3(2,0,0));
glUseProgram(shaderProgram);
GLint loc = glGetUniformLocation(shaderProgram,"model");
glUniformMatrix4fv(loc,1,GL_FALSE,glm::value_ptr(model));
```

함수	인자와 기능
glUseProgram(program)	앞으로 사용할 셰이더 프로그램을 활성화합니다.
glGetUniformLocation(program,name)	그 프로그램의 uniform 위치를 GLint로 반환합니다. -1이면 비활성/찾지 못한 경우입니다.
glUniformMatrix4fv(loc,count,transpose,data)	loc에 count개의 행렬을 전달합니다. GLM 행렬 하나이면 count=1, transpose=GL_FALSE, data는 행렬 데이터 주소입니다.
glm::value_ptr(M)	행렬 데이터 주소를 얻습니다. 행렬을 계산하거나 좌표를 바꾸는 함수가 아닙니다.

버텍스 셰이더의 계산

```
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;
// main 안에서:
gl_Position = projection * view * model * vec4(vPos,1.0);
```

vPos=(1,0,0), model이 x로 2 이동, view가 z로 -5 이동이면 뷰까지의 좌표는 (3,0,-5,1)입니다. projection을 곱하면 클립 좌표이며 마지막 w가 1이 아닐 수 있습니다. 원근 나누기와 뷰포트 매핑은 이후 단계입니다.

계산 결과의 종류를 구별

- **glDrawArrays(GL_TRIANGLES,0,6)**: 6개 정점을 사용해 2개 삼각형을 구성합니다. 위치 변환은 활성 셰이더와 그 uniform에 따라 실행됩니다.
- **glDrawElements(GL_TRIANGLES,6,GL_UNSIGNED_INT,0)**: EBO 인덱스 6개를 사용합니다. 고유 정점 수가 6이라는 뜻은 아닙니다.
- **glEnable(GL_DEPTH_TEST)**: 깊이 비교를 켜는 상태 설정입니다. **glClear**: 버퍼를 지웁니다. 이 함수들은 이동행렬을 반환하거나 좌표에 값을 더하지 않습니다.

F12 함수 중심 예상 연습

실제 기출이 아닌 보충 문제 | 기능 설명과 계산을 함께 쓰세요.

아래 코드에서 I는 glm::mat4(1.0f), p는 동차 점입니다. rotate의 z는 glm::vec3(0,0,1), 90도는 glm::radians(90.0f)로 읽습니다. 식으로 줄여 쓴 인자는 실제 C++ 타입에 맞게 작성합니다.

번호	코드 또는 조건	요구
1	M=translate(I,(2,-1,3)); p=(1,4,-2,1)	함수 인자 설명과 M*p. p의 w=0이면?
2	M=scale(I,(0.5,2,-1)); p=(4,3,2,1)	각 배율 의미와 최종 좌표.
3	M=rotate(I,90도,z); p=(3,-2,1,1)	회전축·각도 설명과 결과.
4	M=translate(I,(2,0,0)); M=rotate(M,90도,z); p=(1,1,0,1)	행렬 곱을 적고 실제 적용 순서·결과.
5	M=rotate(I,90도,z); M=translate(M,(2,0,0)); p=(1,1,0,1)	4번과 결과가 왜 다른가?
6	q=(1,1,0) 중심으로 p=(3,1,0)를 90도 회전	GLM 누적 호출 순서와 결과.
7	V=lookAt((0,0,4),(0,0,0),(0,1,0)); p=(2,3,1,1)	각 인자의 뜻과 뷰 좌표.
8	P=ortho(-4,4,-2,2,1,9); 뷰 점=(2,1,-5)	투영 종류와 NDC.
9	P=perspective(90도,1,1,9); 뷰 점=(1,2,-4)	x/y/w 클립 좌표와 x/y NDC.
10	glViewport(20,30,800,400); NDC=(0.5,-0.5)	네 인자의 뜻과 화면 좌표.
11	v=(0,6,8); N=(0,0,1)	length(v), normalize(v), dot(N,normalize(v)).
12	model=translate(I,(1,0,0)); view=translate(I,(0,0,-3)); vPos=(2,1,0)	셰이더의 view*model*vec4(vPos,1) 값. model을 만들기만 하고 uniform으로 보내지 않았다면?

답안 형식

"함수는 무엇을 하는가 → 각 인자가 무슨 뜻인가 → 어떤 행렬/상태가 만들어지는가 → 점에는 어떤 순서로 적용되는가 → 결과는 어느 좌표계의 값인가" 순서로 적으면 계산의 근거가 드러납니다.

F13 함수 중심 문제 해설

F12의 정답과 핵심 이유

기본 변환 1~3

- **1.** 이동량은 x+2,y-1,z+3. 점 결과=(3,3,1,1). 방향 w=0이면 이동되지 않아 (1,4,-2,0).
- **2.** x 절반, y 두 배, z 반전. 결과=(2,6,-2,1). 원점 기준 신축이며 w=1은 유지됩니다.
- **3.** +z축의 양의 방향 주위 90도. (x,y)→(-y,x)이므로 (2,3,1,1). angle은 실제 호출에서 π/2라디안입니다.

합성과 기준점 4~6

- **4.** M=TR. 먼저 회전: (1,1)→(-1,1), 다음 이동 → **(1,1,0,1)**.
- **5.** M=RT. 먼저 이동: (1,1)→(3,1), 다음 회전 → **(-1,3,0,1)**. 이동과 회전은 일반적으로 교환되지 않습니다.
- **6.** translate(I,q) → rotate(M,90도,z) → translate(M,-q). 실제 계산은 (3,1)→(2,0)→(0,2)→**(1,3)**.

카메라·투영·화면 7~10

- **7.** eye=(0,0,4), 목표점 원점, 기준 위쪽 +y. 기본 방향의 카메라에서 z-4이므로 **(2,3,-3,1)**.
- **8.** 직각 투영. xndc=2/4=0.5, yndc=1/2=0.5. zndc=-2(-5)/8-10/8=0. 따라서 **(0.5,0.5,0)**.
- **9.** c=1,aspect=1. xclip=1,yclip=2,wclip=4. NDC x=0.25,y=0.5. 클립 좌표를 바로 화면 좌표로 쓰지 않습니다.
- **10.** (20,30)은 왼쪽 아래,800과400은 너비·높이. x=20+1.5×400=620, y=30+0.5×200=130. **(620,130)**.

벡터와 GPU 연결 11~12

- **11.** 길이=√(36+64)=10. 단위벡터=(0,0.6,0.8). 내적=0.8. 정규화 전 dot(N,v)=8을 코사인으로 쓰면 안 됩니다.
- **12.** model 적용 (3,1,0,1), view 적용 **(3,1,-3,1)**. CPU에서 만든 행렬은 전송 없이 자동 반영되지 않습니다. 이전 uniform이 유지되거나, 한 번도 설정하지 않은 활성 uniform이면 초기값에 따라 다른 결과가 납니다. 새 코드의 행렬이 적용됐다고 가정할 수 없습니다.

시험 전 마지막 확인: translate·rotate·scale은 행렬을 반환하고, lookAt·ortho·perspective도 행렬을 만듭니다. glViewport·glUseProgram·glEnable은 상태를 설정합니다. 함수의 반환값과 GPU 상태를 분리해서 설명하세요.